

Tri-Level Hybrid DDQN-ALNS: A Hierarchical Learning-Augmented Metaheuristic for the Vehicle Routing Problem with Time Windows

HUYNH NHAT HUY¹, HO THI LINH², NGUYEN NHAT HUY³, AND NGUYEN THI BAO TRAN⁴

¹Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam (e-mail: huynhnhathuy@tdtu.edu.vn)

²Natural Language Processing and Knowledge Discovery Research Group, Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam (e-mail: hotlinh@tdtu.edu.vn)

³Mindx, Ho Chi Minh City, Vietnam (e-mail: huynguyenwork14@gmail.com)

⁴Vinsmart future, Ho Chi Minh City, Vietnam (e-mail: tranntb.se@gmail.com)

Corresponding author: Thi-Linh Ho (e-mail: hothilinh@tdtu.edu.vn).

This research is supported by Ton Duc Thang University.

ABSTRACT The Vehicle Routing Problem with Time Windows (VRPTW) is a fundamental combinatorial optimization problem in logistics, characterized by a hierarchical objective that prioritizes minimizing the total number of deployed vehicles (N_V) before minimizing total travel distance (D_T). While classical Adaptive Large Neighborhood Search (ALNS) remains a widely used and competitive metaheuristic paradigm for vehicle routing, baseline implementations relying on stateless heuristic score aggregation and static acceptance criteria can encounter search stagnation in complex spatial-temporal topologies. Conversely, pure end-to-end Deep Reinforcement Learning (DRL) methods face feasibility challenges under hard time windows and out-of-distribution generalization barriers. To bridge these paradigms, this paper presents Tri-Level Hybrid DDQN-ALNS, a learning-augmented optimization framework structured into three coordinated tiers: (i) a Macro Decision Tier (Plateau Controller) formulated as an Approximate Semi-Markov Decision Process over an Aggregated State Space (Aggregated SMDP) to monitor search stagnation and govern regime transitions, (ii) a Micro Selection Tier where a Dueling Double Deep Q-Network (DDQN) with an explicit normalized Softmax-Entropy policy-concentration gate learns state-conditioned destroy-repair pairings, complemented by a Learned Acceptance Criterion (LAC) that operates as an online delayed-outcome predictive acceptance override over an $H_{lac} = 80$ lookahead horizon without vetoing valid Simulated Annealing acceptances, and (iii) a Heuristic Execution Muscle leveraging $O(1)$ incremental forward-time-slack feasibility queries per candidate insertion, a Contrastive Graph Neural Network (GNN) spatial edge predictor to prune unpromising insertion arcs, polynomial Generalized Ejection Chains (GEC), and periodic time-limited Set Partitioning recombination over elite routes via HiGHS with certified lexicographic penalty bounds. We evaluate the proposed framework across 74 standard Solomon-100, Homberger-200, and Homberger-400 benchmark instances under strictly isolated cold-start execution protocols. Tri-Level Hybrid DDQN-ALNS achieves statistically significant vehicle fleet reductions (31 Wins / 40 Ties / 3 Losses, Wilcoxon $p = 6.91 \times 10^{-7}$); on the 54.1% fleet-matched subset ($N_{matched} = 40/74$), it secures an average -3.73% travel distance reduction ($p = 9.31 \times 10^{-8}$) while strictly preserving 100% operational constraint feasibility.

INDEX TERMS Adaptive Large Neighborhood Search, Deep Reinforcement Learning, Double DQN, Semi-Markov Decision Process, Metaheuristics, Vehicle Routing Problem with Time Windows.

I. INTRODUCTION

The Vehicle Routing Problem with Time Windows (VRPTW) is a foundational NP-hard combinatorial optimization

tion problem underpinning modern freight logistics, last-mile parcel delivery, and sustainable distribution networks [1], [2]. In addition to the combinatorial explosion inherent in classical routing, VRPTW enforces hard customer time-window constraints: each vehicle route must simultaneously respect vehicle capacity limits and temporal feasibility at every stop.

In commercial freight operations, the optimization objective is strictly lexicographic: dispatchers prioritize minimizing the total number of deployed vehicles (N_V) before minimizing total travel distance (D_T) [3], [4]. Formally, letting $\mathcal{S}$ denote the space of feasible routing solutions, a solution $s_1 \in \mathcal{S}$ is strictly preferred over $s_2 \in \mathcal{S}$ (denoted $s_1 \prec s_2$) if and only if:

$$NV(s_1) < NV(s_2) \vee (NV(s_1) = NV(s_2) \wedge TD(s_1) < TD(s_2)). \quad (1)$$

This hierarchical ordering reflects the economic structure of commercial transportation, where fixed vehicle capital acquisition, maintenance, insurance, and driver shift wages dominate marginal fuel consumption.

Adaptive Large Neighborhood Search (ALNS) [5], [6] has long served as a leading metaheuristic paradigm for VRPTW, exploring diverse neighborhood structures via iterative ruin-and-recreate operations while strictly preserving feasibility. However, standard ALNS implementations rely on stateless roulette-wheel or multi-armed bandit mechanisms that aggregate historical reward scores across coarse segments. This design exhibits two structural weaknesses: (i) **Stateless Operator Selection**, ignoring instantaneous search state such as spatial route fragmentation, temporal slack, or capacity utilization; and (ii) **Myopic Plateau Stagnation**, lacking multi-step coordination to escape structural plateaus where vehicle elimination requires evaluating temporarily non-improving intermediate states.

In parallel, Neural Combinatorial Optimization (NCO) has emerged as a data-driven alternative, using Deep Reinforcement Learning (DRL) models such as Pointer Networks [7], Attention Models [8], and constructive policies [9]. While capturing spatial regularities without manual heuristic engineering, direct application of pure end-to-end neural solvers to industrial VRPTW encounters critical bottlenecks: (i) **Feasibility Fragility under Tight Time Windows**, where autoregressive decoding deadlocks or requires delicate penalty tunings [10], [11]; (ii) **Objective Misalignment**, optimizing scalarized distance rather than strict lexicographic fleet minimization ($N_V \succ D_T$); and (iii) **Out-of-Distribution Sensitivity**, suffering steep performance collapse on larger scales or clustered customer distributions without expensive retuning.

To bridge these paradigms, this paper presents Tri-Level Hybrid DDQN-ALNS, a learning-augmented optimization framework that embeds state-conditioned reinforcement learning directly inside the iterative search loop of ALNS. Rather than replacing the metaheuristic with an opaque neural solver, our approach maintains hard operational constraint

feasibility and interpretable neighborhood operators from operations research, structuring search governance into three coordinated tiers: two learned reinforcement learning tiers (Macro Plateau Governance and Micro Operator Selection) coupled with a high-throughput heuristic execution tier.

The primary contributions of this paper are summarized as follows:

- **Three-Tier Hierarchical Control Architecture:** We formulate metaheuristic search management as an Approximate Semi-Markov Decision Process over an Aggregated Feature Space (Aggregated SMDP), decoupling macro-level exploration regimes (Plateau Controller) from micro-level destroy-repair operator selection (Operator Controller).
- **State-Conditioned Micro Selection with Policy-Concentration Gating:** We develop a Dueling Double Deep Q-Network (DDQN) over a 20-dimensional spatiotemporal state space, using a normalized Softmax-Entropy policy-concentration gate to adaptively blend learned neural Q-values with empirical variance-aware UCB bandit priors.
- **Learned Acceptance Criterion (LAC) as Predictive Override:** We introduce a supervised neural acceptance model trained on delayed downstream lookahead signals ($H_{\text{lac}} = 80$), augmenting Simulated Annealing with state-aware predictive lookahead to rescue trapped exploratory trajectories during deep cooling without vetoing valid SA acceptances.
- **Spatial GNN Edge Guidance and High-Throughput Search:** We combine an offline-trained Contrastive Graph Neural Network (GNN) and granular candidate restriction that prunes 97.5% – 98.7% of unpromising insertion arcs with $O(1)$ forward-slack feasibility testing and time-limited Set Partitioning integer programming recombination via HiGHS.
- **Rigorous Scale-Aware Benchmark across 74 Instances:** We evaluate the framework across 74 standard benchmark instances (56 Solomon-100, 12 Homberger-200, and 6 Homberger-400) under strictly isolated independent cold-starts, providing an empirical characterization of solution quality, fleet reduction, and computational trade-offs supported by non-parametric statistical hypothesis testing.

The remainder of this paper is organized as follows: Section II reviews related literature. Section III presents the problem formulation. Section IV describes the proposed Tri-Level Hybrid DDQN-ALNS methodology. Section V details the empirical benchmark results, statistical tests, and ablation analyses. Section VI discusses scale-aware dynamics, analytical properties, and limitations. Finally, Section VII concludes the paper.

II. RELATED WORK

TABLE 1. Taxonomy of Representative Methodologies for VRPTW and Related Routing Problems

Methodology Category	Representative Works	Hard TW Feasible	Objective Paradigm	State-Aware Selection	Hierarchical Control	Learned Accept	Per-Step Compute Cost
Classical Metaheuristics	ALNS [5], [6]	✓	Lexicographic	× (Stateless)	×	× (SA)	Minimal (Heuristic)
	SISR [12], HGS [13]	✓	Lexicographic	× (Rule-based)	×	× (SA/Thresh)	Minimal (Heuristic)
End-to-End Neural Solvers	AM [8], DRL-VRP [9]	× (Penalized)	Scalarized	N/A	×	N/A	Very Low (One-shot)
	Neural VRPTW [10], [11]	× (Mask-fragile)	Scalarized	N/A	×	N/A	Very Low (One-shot)
Single-Agent Hybrid RL-LNS	Learned LNS [14], [15]	✓	Scalarized	✓	× (Single-agent)	× (SA)	Moderate (Policy NN)
	Neural LNS [16]	✓	Scalarized	✓	× (Single-agent)	× (Greedy)	Moderate (Policy NN)
Proposed Tri-Level Hybrid DDQN-ALNS	This Work	✓	Lexicographic	✓	✓ (Dual HMDP)	✓ (LAC)	High (1.5–4 × RL/MIP)

A. CLASSICAL METAHEURISTICS AND ALNS FOR VRPTW

Metaheuristics have long dominated practical VRPTW optimization [4]. Building on Solomon’s insertion heuristics [2] and Shaw’s Large Neighborhood Search [17], Ropke and Pisinger [5], [6], [18] formalized Adaptive Large Neighborhood Search (ALNS), wherein removal and insertion operators compete for selection via roulette-wheel probabilities updated from historical performance. For fleet minimization, Guided Ejection Search [19] and penalty-based edge assembly memetic algorithms [20] represent dedicated mechanisms, while matheuristic hybrids leverage elite route pools to periodically solve Set Partitioning models [13], [21]–[23]. More recently, Slack Induction by String Removals [12] demonstrated that specialized ruin operators can rival complex frameworks.

Despite this success, standard ALNS has an intrinsic limitation: operator weights are updated as exponentially smoothed scores across coarse iteration segments, resulting in delayed, aggregated credit assignment. The selection policy is stateless, ignoring route topology, cluster fragmentation, and time-window tightness. Turkeš et al. [24] showed that this adaptive mechanism yields only marginal gains over non-adaptive LNS, confirming that the operator-selection policy is the key bottleneck.

B. NEURAL COMBINATORIAL OPTIMIZATION AND END-TO-END DEEP RL

Neural Combinatorial Optimization (NCO) learns data-driven routing policies using Pointer Networks [7], [25], reinforcement learning on dynamic customer states [9], Attention Models [8], and Graph Neural Networks [26], [27]. Extending NCO to VRPTW [10], [11] achieves rapid inference on small instances but encounters severe operational hurdles: (i) Feasibility Fragility, where hard time windows cause constructive decoding deadlocks; (ii) Lexicographic Misalignment, training on scalarized surrogate costs rather than the primary-secondary fleet hierarchy ($N_V \succ D_T$); (iii) Out-of-Distribution Degradation, collapsing on larger scales ($N \geq 200$) or heterogeneous topologies; and (iv) Lack

of Auditability, functioning as an opaque black box unsuitable for human dispatcher intervention.

C. LEARNING-AUGMENTED METAHEURISTICS

To combine machine learning with operational guarantees, the “Learning to Optimize” paradigm [28] embeds neural models inside metaheuristic search loops for operator selection [14], [29], solution perturbation [30], and learned destroy policies [15], [16]. However, existing hybrids suffer from three key deficiencies: (i) Single-Level Architectures, deploying a flat policy across all search stages without decoupling macro exploration regimes from micro operator selection; (ii) Static Acceptance, relying on standard Simulated Annealing that evaluates moves solely on instantaneous cost degradation ΔC ; and (iii) Scalarized Focus, lacking mechanisms tailored to lexicographic fleet reduction.

D. POSITIONING OF THIS WORK

Table 1 situates Tri-Level Hybrid DDQN-ALNS within the routing literature. Our approach combines hard constraint guarantees with a dual-agent Hierarchical MDP and a Learned Acceptance Criterion. Macro/micro decoupling addresses a fundamental credit-assignment challenge across timescales [43]: rewards for escaping multi-hundred-iteration plateaus are temporally distant from single destroy-repair decisions, and folding both into one policy degrades gradient specialization. The macro tier’s semi-MDP formulation over $\Delta_{\text{seg}} = 100$ -iteration options isolates plateau governance from micro operator selection. Table 1 also highlights the operational trade-off: our framework accepts higher per-iteration computational overhead ($1.5 \times -4 \times$ per-step slowdown from online RL updates and MIP solves) to achieve superior asymptotic search quality and escape deep combinatorial plateaus.

III. PROBLEM FORMULATION

The Vehicle Routing Problem with Time Windows (VRPTW) is formally defined on a complete directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{A})$. The vertex set $\mathcal{V} = \mathcal{N} \cup \{0, n + 1\}$ comprises customer nodes $\mathcal{N} = \{1, 2, \dots, n\}$, origin depot node 0,

and destination depot node $n + 1$. The physical depot is modeled as two separate nodes (0 and $n + 1$) to represent route departure and return without self-loops. The arc set is $\mathcal{A} = \{(i, j) : i, j \in \mathcal{V}, i \neq j, i \neq n + 1, j \neq 0\}$.

Each customer $i \in \mathcal{N}$ has a non-negative demand $q_i > 0$, service duration $s_i \geq 0$, and service time window $[e_i, l_i]$, where e_i and l_i denote the earliest and latest allowable service start times, respectively. Depot nodes satisfy $q_0 = q_{n+1} = 0$, $s_0 = s_{n+1} = 0$, and $[e_0, l_{n+1}]$ defines the global planning horizon. Each arc $(i, j) \in \mathcal{A}$ has travel distance $c_{ij} \geq 0$ and travel time $t_{ij} \geq 0$. A homogeneous fleet of K vehicles, denoted by $\mathcal{K} = \{1, \dots, K\}$, is stationed at the depot, with each vehicle possessing carrying capacity Q .

A. MATHEMATICAL MODEL AND DECISION VARIABLES

The optimization model employs binary routing variables $x_{ij}^k \in \{0, 1\}$ (1 if vehicle $k \in \mathcal{K}$ traverses arc $(i, j) \in \mathcal{A}$, and 0 otherwise) and continuous schedule variables $w_i^k \geq 0$ specifying the time at which vehicle $k \in \mathcal{K}$ begins service at node $i \in \mathcal{V}$. The primary objective is lexicographic: first minimizing the number of deployed vehicles (N_V), and secondarily minimizing total travel distance (D_T):

$$\text{lex min} \left(\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{N}} x_{0j}^k, \sum_{k \in \mathcal{K}} \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}^k \right) \quad (2)$$

In computational solver implementations, this hierarchy is evaluated via a scalarized surrogate objective using a large penalty $\mathcal{M} \gg \sum_{(i,j) \in \mathcal{A}} c_{ij}$:

$$\min \sum_{k \in \mathcal{K}} \left(\mathcal{M} \sum_{j \in \mathcal{N}} x_{0j}^k + \sum_{(i,j) \in \mathcal{A}} c_{ij} x_{ij}^k \right) \quad (3)$$

subject to:

$$\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{V}, j \neq i} x_{ij}^k = 1, \quad \forall i \in \mathcal{N} \quad (4)$$

$$\sum_{j \in \mathcal{N} \cup \{n+1\}} x_{0j}^k \leq 1, \quad \sum_{j \in \mathcal{N} \cup \{0\}} x_{j,n+1}^k = \sum_{j \in \mathcal{N} \cup \{n+1\}} x_{0j}^k, \quad \forall k \in \mathcal{K} \quad (5)$$

$$\sum_{i \in \mathcal{V}, i \neq h} x_{ih}^k - \sum_{j \in \mathcal{V}, j \neq h} x_{hj}^k = 0, \quad \forall h \in \mathcal{N}, k \in \mathcal{K} \quad (6)$$

$$\sum_{i \in \mathcal{N}} q_i \sum_{j \in \mathcal{V}, j \neq i} x_{ij}^k \leq Q, \quad \forall k \in \mathcal{K} \quad (7)$$

$$w_i^k + s_i + t_{ij} - M_{ij}(1 - x_{ij}^k) \leq w_j^k, \quad \forall (i, j) \in \mathcal{A}, k \in \mathcal{K} \quad (8)$$

$$e_i \leq w_i^k \leq l_i, \quad x_{ij}^k \in \{0, 1\}, \quad w_i^k \geq 0, \quad \forall i \in \mathcal{V}, (i, j) \in \mathcal{A}, k \in \mathcal{K} \quad (9)$$

Constraint (4) ensures that every customer is visited exactly once. Constraints (5) enforce that each vehicle departs from origin depot 0 and arrives at destination depot $n + 1$

at most once. Flow conservation across all customer nodes is guaranteed by (6). Constraint (7) prevents vehicle capacity violations. Schedule propagation is maintained via (8), where $M_{ij} = \max\{0, l_i + s_i + t_{ij} - e_j\}$ defines the tightest valid Big- M upper bound. Customer time-window compliance and domains are governed by (9); if a vehicle arrives at customer i prior to e_i , it incurs an idle waiting time such that service begins at $w_i^k = e_i$. While w_i^k is indexed per vehicle for strict MIP formulation alignment with x_{ij}^k , constraint (4) ensures that exactly one vehicle serves each customer i , rendering only one w_i^k active in any feasible dispatch plan.

IV. PROPOSED METHODOLOGY

This section details the proposed Tri-Level Hybrid DDQN-ALNS framework. The architecture integrates online adaptive reinforcement learning decision layers with operations research search mechanics to optimize the lexicographic VRPTW objective while maintaining operational constraint feasibility.

A. TRI-LEVEL ARCHITECTURE OVERVIEW

The framework decomposes the global routing search into three coordinated operational tiers, illustrated in Fig. 1:

The framework orchestrates search across three coordinated tiers: (1) **Macro Decision Tier (Plateau Controller π_{macro})**: formulated as an Aggregated semi-MDP over segments of length $\Delta_{\text{seg}} = 100$, where a Dueling Double DQN [40], [41] with Prioritized Experience Replay [42] evaluates convergence velocity and transitions among 7 exploration/recombination regimes; (2) **Micro Selection Tier (Operator Controller π_{micro} & LAC)**: operates per iteration t , using a Micro Dueling DDQN with normalized Softmax-Entropy policy-concentration gating to select composite destroy-repair operator pairings conditioned on a 20-dimensional state vector, augmented by a Learned Acceptance Criterion (LAC) evaluating delayed downstream lookahead; and (3) **Heuristic Search Tier (Granular ALNS & Route-Pool Recombination)**: executes ruin-and-recreate operations with $O(1)$ Savelsbergh forward-slack feasibility testing, restricts moves to top- k ($k = 25$) spatiotemporally correlated candidate neighbors, and periodically solves a time-limited Set Partitioning formulation over elite routes via HHS (4.0s wall-clock cutoff).

B. MACRO LEVEL: PLATEAU CONTROLLER (AGGREGATED SMDP FORMULATION)

We formulate the macro search governance tier as a Semi-Markov Decision Process [43] over an Aggregated State Representation (Aggregated SMDP) defined by the tuple $(\mathcal{S}_{\text{macro}}, \mathcal{A}_{\text{macro}}, P_{\text{macro}}, R_{\text{macro}}, \gamma_{\text{macro}})$. Because the underlying combinatorial route pool and fine-grained trajectory history are projected into a finite 13-dimensional feature space $\mathbf{s}_k^{\text{macro}}$, the decision tier operates over a compact aggregated state representation with temporal options of fixed duration $\Delta_{\text{seg}} = 100$ low-level ALNS iterations. The temporally extended transition kernel $P_{\text{macro}}(\mathbf{s}_{k+1}^{\text{macro}} | \mathbf{s}_k^{\text{macro}}, a_k)$ represents

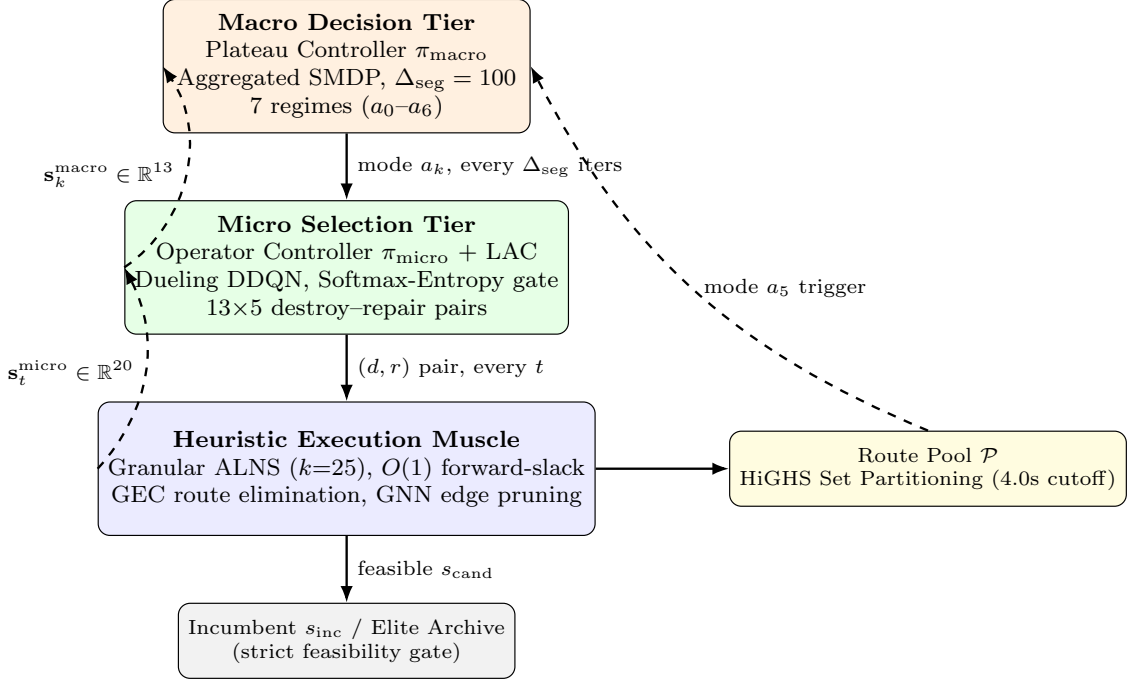


FIGURE 1. Tri-Level Hybrid DDQN-ALNS control architecture. Solid arrows: downward control flow. Dashed arrows: upward state feedback consumed by the RL training loops.

the cumulative transition distribution over the 100 underlying stochastic destroy-repair steps, with effective discount factor $\gamma_{\text{macro}} = \gamma^{\Delta_{\text{seg}}}$.

1) Macro State Representation

At the start of segment k , the macro controller observes a 13-dimensional state vector $\mathbf{s}_k^{\text{macro}} \in \mathbb{R}^{13}$, capturing convergence velocity, fleet economics, and solution topology:

$$\mathbf{s}_k^{\text{macro}} = \left[\min\left(\frac{\text{no_imp}}{\text{patience}}, 1\right), \min\left(\frac{C_{\text{cur}} - C_{\text{best}}}{C_{\text{best}}}, 1\right), \min\left(\frac{T}{T_0}, 1.5\right), \lambda_{\text{imp_rate}}, \min\left(\frac{NV_{\text{cur}}}{NV_{\text{init}}}, 2\right), \frac{NV_{\text{cur}} - NV_{\text{best}}}{\max(NV_{\text{init}}, 1)}, rb, lb, \tau_{\text{tight}}, \bar{s}, \rho_{\text{fill}}, \rho_{\text{pool}}, \lambda_{\text{fleet}} \right] \quad (10)$$

where no_imp is the number of consecutive unimproved iterations, T/T_0 is the normalized annealing temperature, $rb \in [0, 1]$ and $lb \in [0, 1]$ measure spatial route separation and capacity balance, $\tau_{\text{tight}} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}(l_i - e_i < 0.5 \cdot T_{\text{depot}})$ represents the fraction of tight time-window customers relative to the depot operational horizon $T_{\text{depot}} = l_{n+1} - e_0$, $\bar{s}$ is average forward time slack, ρ_{fill} is average vehicle capacity utilization, and $\rho_{\text{pool}} = \min(|\mathcal{P}|/\mathcal{P}_{\text{max}}, 1.0)$ tracks route pool fill ratio.

2) Macro Action Space & Search Regimes

The macro action space $\mathcal{A}_{\text{macro}}$ comprises 7 distinct search modes ($|\mathcal{A}_{\text{macro}}| = 7$): a_0 (DEFAULT): balanced exploration; a_1 (INTENSE_EXPLOIT): deep cooling with intensified local search ($\gamma_{\text{destroy}} \leftarrow 0.7\gamma_0$); a_2

(DIVERSIFY): thermal reheating ($T \leftarrow 1.5T$) with elevated destroy ratios ($\gamma_{\text{destroy}} \leftarrow 1.5\gamma_0$); a_3 (TW_RESCUE): targeted ruin-and-recreate on tight-window clusters; a_4 (ROUTE_ELIMINATE): route-depletion via Generalized Ejection Chains; a_5 (POOL_RECOMBINE): time-limited Set Partitioning over Route Pool $\mathcal{P}$ via HiGHS (4.0s cutoff); and a_6 (INFEASIBLE_DESCENT): controlled temporal and capacity violation relaxation governed by an adaptive Lagrangian penalty objective $C_{\text{pen}}(s) = C(s) + \lambda_{\text{tw}} \sum_i \max(0, a_i - l_i) + \lambda_{\text{cap}} \sum_r \max(0, q(r) - Q)$, with dynamic multipliers $\lambda_{\text{imp_rate}}, \lambda_{\text{fleet}} \in [0, 1, 100.0]$.

3) Multi-Objective Segment Reward & Surrogate Policy Optimization

To guide reinforcement learning exploration in alignment with the primary-secondary optimization hierarchy, the segment reward R_{seg} balances vehicle reduction gains against distance savings via an adaptive fleet pressure factor $\lambda_{\text{fleet}} \in [0, 1]$:

$$R_{\text{seg}} = \text{Base} + \lambda_{\text{fleet}} \Psi_{NV} + (1 - \lambda_{\text{fleet}}) \Psi_{TD} + \text{Bonus}_{NV} + \Delta\Phi \quad (11)$$

where $\Psi_{NV} = \sum_{j=1}^4 \beta_{NV,j} \Delta X_j$ and $\Psi_{TD} = \sum_{j=1}^4 \beta_{TD,j} \Delta X_j$ linearly weight improvements across best-so-far and incumbent fleet size (ΔNV) and travel distance (ΔC), with representative primary fleet coefficient $\beta_{NV,1} = 8.0$. Baseline computational penalty Base penalizes intensive local search passes and unproductive stagnation, while Bonus_{NV} provides discrete milestone reinforcement (+15.0) upon physical vehicle elimination. Complete parameter listings and definitions are tabulated in Supplementary

Material Table S1.

The presence of two parallel fleet-reward pathways in (11) addresses credit assignment across temporal scales: (i) **Dense Continuous Guidance** ($\lambda_{\text{fleet}} \Psi_{NV}$) encourages moves that balance inter-route capacity utilization and narrow the vehicle count gap relative to the incumbent; and (ii) **Sparse Milestone Reinforcement** (Bonus_{NV}) awards +15.0 exclusively upon physical vehicle elimination, preventing the sparse route collapse signal from being diluted across routine distance variations.

a: Experimental Firewall and Reward Calibration

All reward constants in (11)–(13) and structural parameters were calibrated on independent synthetic pilot topologies generated via domain randomization during initial prototyping (detailed in Supplementary Material Section 9 and Table S1). To prevent data leakage and hyperparameter overfitting, a strict experimental firewall was enforced: zero reward tuning, grid search, or post-hoc parameter adjustments were conducted on any of the 74 benchmark test instances (56 Solomon-100, 18 Gehring-Homberger). All test instances were evaluated strictly out-of-sample under isolated cold-starts.

We emphasize a fundamental architectural dichotomy between the combinatorial evaluation layer and the reinforcement learning training layer: (1) **Strict Lexicographic Evaluation**: Incumbent replacement, elite archive admission, and final solution selection strictly evaluate the true lexicographic ordering $f(s) = (NV(s), TD(s))$, guaranteeing that fleet reduction strictly dominates distance; and (2) **Smooth RL Reward Surrogate**: The reinforcement learning policy receives scalarized reward feedback R_{seg} modulated by λ_{fleet} . When surplus capacity exists ($NV_{\text{cur}} > NV_{\text{best}}$), setting $\lambda_{\text{fleet}} \rightarrow 1$ concentrates gradients on fleet collapse; once fleet size stabilizes ($NV_{\text{cur}} = NV_{\text{best}}$), setting $\lambda_{\text{fleet}} \leq 0.5$ transfers gradient sensitivity to routing distance.

To accelerate learning while preventing artificial reward cycles in the surrogate MDP, we incorporate potential-based reward shaping $\Delta\Phi = \gamma_{\text{macro}} \Phi(s_{k+1}^{\text{macro}}) - \Phi(s_k^{\text{macro}})$. We explicitly note that this term implements potential-based shaping of the macro surrogate objective [31] rather than guaranteeing policy invariance with respect to the underlying lexicographic VRPTW objective (which is enforced externally by the combinatorial evaluator). The adaptive fleet pressure factor $\lambda_{\text{fleet}} \in [0, 1]$ dynamically shifts optimization priority toward fleet reduction whenever surplus capacity exists:

$$\lambda_{\text{fleet}}(s) = \sigma\left(8.0 \cdot \frac{NV_{\text{cur}} - NV_{\text{best}}}{NV_{\text{init}}}\right) \quad (12)$$

where $\sigma(\cdot)$ is the logistic sigmoid, with the state potential

defined as:

$$\Phi(s) = -\lambda_{\text{fleet}}(s) \frac{\max(NV - NV_{\text{best}}, 0)}{NV_{\text{init}}} - (1 - \lambda_{\text{fleet}}(s)) \cdot 0.5 \text{clip}\left(\frac{TD - TD_{\text{best}}}{TD_{\text{best}}} \cdot 100, -25, 25\right) \quad (13)$$

The Macro Plateau Controller trains a dedicated Dueling Double DQN network (Q_{macro}) with prioritized replay buffer $\mathcal{D}_{\text{macro}}$, executing Adam gradient steps with Cosine Annealing learning rate scheduling at the completion of each macro segment.

C. MICRO LEVEL: OPERATOR CONTROLLER & ACTION BLENDING

1) Micro State Representation

At each iteration t , the operator controller observes a 20-dimensional state vector $s_t^{\text{micro}} \in \mathbb{R}^{20}$ (Table 2), detailing fine-grained search dynamics and route topology.

2) Composite Action Space

The micro action space $\mathcal{A}_{\text{micro}}$ comprises $N_D \times N_R = 13 \times 5 = 65$ composite heuristic pairs, combining **13 Destroy Operators** (Random, Worst-cost, Shaw spatiotemporal relatedness, Route portion removal, Time-window urgency removal, Route-Depletion Destroy, Route dispersion elimination, Costly route elimination, Cross-route Shaw, Route merge sampling, Route absorb disruption, Neural worst, Neural Shaw) with **5 Repair Operators** (Greedy insertion, 2-Regret, 3-Regret, Time-window greedy, Forward-time-slack greedy).

3) Dueling DDQN Architecture & Policy-Concentration Blending

The Micro Q-network adopts a Dueling architecture [32] with layer normalization:

$$Q(s, a; \theta) = V(s; \theta_v) + \left(A(s, a; \theta_a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta_a) \right) \quad (14)$$

To adaptively balance learned neural predictions with exploration priors, we implement a normalized Softmax-Entropy Policy-Concentration Gate. Let $\pi_{\theta}(a | s) = \text{softmax}(Q(s, a)/\tau_{\text{soft}})$ be the Boltzmann policy at softmax

TABLE 2. 20-Dimensional Micro State Vector s_t^{micro}

Idx Sym	Definition	Idx Sym	Definition
$s_1 \Delta C_{\text{rel}}$	$(C_{\text{cur}} - C_{\text{best}})/C_{\text{best}}$	$s_{11} \rho_{\text{fill}}$	Capacity fill ratio
$s_2 \text{Ratio}_{NV}$	$NV_{\text{cur}}/NV_{\text{init}}$	$s_{12} \rho_{\text{pool}}$	Pool fill ratio $ \mathcal{P} / \mathcal{P}_{\text{max}} $
$s_3 \text{Prog}_{\text{global}}$	t/t_{max}	$s_{13} m_{\text{idx}}$	Active macro mode index
$s_4 \text{Prog}_{\text{seg}}$	$(t \bmod \Delta_{\text{seg}})/\Delta_{\text{seg}}$	$s_{14} \Delta NV_{\text{rel}}$	$(NV_{\text{cur}} - NV_{\text{best}})/NV_{\text{init}}$
$s_5 T_{\text{norm}}$	T/T_0	$s_{15} \text{Imp}_{\text{rec}}$	Recent improvement rate
$s_6 \text{Stag}_{\text{ratio}}$	$\text{no_imp}/\text{patience}$	$s_{16} \text{Gini}_{\text{len}}$	Gini of route lengths
$s_7 r_b$	Route separation index	$s_{17} \mu_{\text{slack}}$	Mean customer forward slack
$s_8 lb$	Capacity load balance	$s_{18} \sigma_{\text{slack}}$	Std of forward slacks
$s_9 \tau_{\text{tight}}$	Frac tight TW customers	$s_{19} \text{Frac}_{\text{cap}}$	Fraction routes $> 90\%$ cap
$s_{10} \bar{s}$	Normalized mean slack	$s_{20} \text{Var}_{\text{dist}}$	Variance of route centroids

temperature $\tau_{\text{soft}} = 1.0$. The normalized Shannon entropy is defined as:

$$\mathcal{H}(\pi_\theta(\cdot | s)) = -\frac{1}{\ln |\mathcal{A}|} \sum_{a \in \mathcal{A}} \pi_\theta(a | s) \ln \pi_\theta(a | s) \in [0, 1] \quad (15)$$

We define the policy-concentration weight as $w_{\text{conf}}(s) = 1.0 - \mathcal{H}(\pi_\theta(\cdot | s))$. High concentration ($w_{\text{conf}} \rightarrow 1$) indicates that the Boltzmann policy is sharply peaked around a dominant operator pair, whereas low concentration ($w_{\text{conf}} \rightarrow 0$) signals an exploratory regime.

To ensure commensurability across diverse score scales, components are standardized: (1) neural Q-values are z-score standardized, $\tilde{Q}(s, a) = (Q(s, a) - \bar{Q}(s)) / (\sigma_Q(s) + \epsilon)$, with instantaneous mean $\bar{Q}(s)$ and standard deviation $\sigma_Q(s)$; (2) heuristic log-priors are min-max normalized, $\tilde{P}(a) \in [0, 1]$; and (3) historical bandit rewards $\bar{\mu}_{\text{bandit}}(a)$ are tracked via Welford updates. The effective action-selection score $Q_{\text{eff}}(s, a)$ dynamically blends neural Q-values with historical priors and the variance-aware UCB bonus adapted from UCB [39] and UCB-V [38], with the exploration bonus explicitly gated by $(1 - w_{\text{conf}}(s))$:

$$Q_{\text{eff}}(s, a) = w_{\text{conf}}(s) \tilde{Q}(s, a) + (1 - w_{\text{conf}}(s)) \left[\alpha_p \tilde{P}(a) + \beta_b \bar{\mu}_{\text{bandit}}(a) + c_{\text{ucb}} \sigma_a \sqrt{\ln N_t / N_a(t)} \right] \quad (16)$$

where $\alpha_p = 0.35$ scales prior log-probabilities, $\beta_b = 0.65$ weights decaying historical posterior rewards $\bar{\mu}_{\text{bandit}}(a)$, $c_{\text{ucb}} = 1.0$ governs the UCB exploration bonus, and $\sigma_a = \sqrt{M_{2,a} / \max(N_a(t) - 1, 1)}$ denotes the online sample standard deviation tracked via Welford's algorithm. Action counters are initialized with pseudocount $N_a(0) = 1$ ($\forall a \in \mathcal{A}$) and $N_t = \sum_a N_a(t)$ for finite numerical bounds.

Gating priors and UCB bonuses by $(1 - w_{\text{conf}}(s))$ ensures that as policy concentration approaches 1 ($w_{\text{conf}} \rightarrow 1$), selection delegates to learned neural Q-values $\tilde{Q}(s, a)$, while smoothly reverting to variance-aware UCB exploration when policy entropy is diffuse ($w_{\text{conf}} \rightarrow 0$).

D. LEARNED ACCEPTANCE CRITERION (LAC): SA WITH PREDICTIVE LOOKAHEAD OVERRIDE

In classical ALNS, move acceptance relies on Simulated Annealing (SA), which accepts improving transitions ($\Delta C < 0$) deterministically and non-improving transitions ($\Delta C \geq 0$) with probability $P_{\text{SA}} = \exp(-\Delta C / T)$. During deep cooling stages ($T \rightarrow 0$), P_{SA} vanishes exponentially, restricting search to downhill moves and causing premature local trapping.

To prevent search entrapment without compromising the convergence stability of Simulated Annealing, the Learned Acceptance Criterion (LAC) operates as an online supervised binary classifier $\phi_{\text{LAC}} : \mathbb{R}^9 \rightarrow [0, 1]$. Rather than serving

as an exclusionary filter or replacement, LAC implements a disjunctive acceptance decision rule:

$$\text{Accept}(s' | s, T) \iff (p_t \geq \tau_{\text{acc}}(t)) \vee (P_{\text{SA}}(\Delta C, T) > U) \quad (17)$$

where $p_t = \phi_{\text{LAC}}(\mathbf{z}_t)$, $U \sim \text{Uniform}(0, 1)$, and $\tau_{\text{acc}}(t) \in [0.40, 0.75]$ is an adaptive confidence threshold.

Crucially, the disjunctive operator ($\vee$) guarantees that LAC acts strictly as an **acceptance booster / rescue mechanism** via two properties: (i) **Preservation of Downhill Transitions**: Any move accepted by standard Simulated Annealing ($P_{\text{SA}} > U$) is unconditionally accepted, preserving classical asymptotic convergence guarantees; and (ii) **Predictive Rescue of Exploratory Transitions**: When Simulated Annealing rejects a non-improving move ($P_{\text{SA}} \leq U$) during cold cooling stages, LAC evaluates whether the topological structural transition is likely to yield a downstream incumbent improvement within lookahead horizon $H_{\text{lac}} = 80$ iterations. If predicted probability exceeds the threshold ($p_t \geq \tau_{\text{acc}}$), the move is accepted as an exploratory rescue step.

Because decision rule (17) operates as an additive booster, it systematically increases the volume of accepted non-improving transitions during deep cooling. In our ablation and limitations analyses (Sections V-F and VI-D), we discuss the methodological implication of distinguishing state-conditioned lookahead intelligence from generic acceptance rate inflation.

1) Feature Representation and Endogenous Training Dynamics

The LAC input feature vector $\mathbf{z}_t \in \mathbb{R}^9$ comprises 9 normalized features:

$$\mathbf{z}_t = \left[\text{sign}(\Delta C) \sqrt{|\Delta C| / C_{\text{best}}}, \frac{T}{T_0}, \frac{\text{no_imp}}{\text{patience}}, \frac{NV_{\text{cur}}}{NV_{\text{init}}}, \Delta NV, \rho_{\text{fill}}, \bar{s}, \frac{t}{T_{\text{max}}}, \exp\left(-\frac{\max(\Delta C, 0)}{T}\right) \right] \quad (18)$$

During the first 300 iterations, the solver operates under standard Simulated Annealing while accumulating state-action transition tuples $(\mathbf{z}_t, s_t, s_{t+1})$ into an online lookahead FIFO buffer. At iteration $t + H_{\text{lac}}$, delayed binary target labels $y_t \in \{0, 1\}$ are assigned based on whether the search trajectory discovered a new best solution within the lookahead horizon ($y_t = 1 \iff \min_{k \in [t+1, t+H_{\text{lac}}]} F(s_k) < F(s_{\text{inc}, t})$).

Because training examples are labeled online from the active search trajectory, the empirical training distribution is endogenous and non-stationary. LAC trains a compact 4-layer MLP ($9 \rightarrow 64 \rightarrow 48 \rightarrow 32 \rightarrow 1$) via online mini-batch Adam gradient descent ($\text{lr} = 10^{-3}$) with class-weighted binary cross-entropy loss:

$$\mathcal{L}_{\text{LAC}}(\phi) = -\frac{1}{B} \sum_{i=1}^B \left[w_1 y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] \quad (19)$$

where $p_i = \phi_{\text{LAC}}(\mathbf{z}_i)$ and $w_1 = N_{\text{neg}} / N_{\text{pos}}$ balances positive discovery event sparsity, bounding inference latency to $< 0.05\text{ms}$ per evaluation.

E. HEURISTIC SEARCH MUSCLE & ALGORITHMIC ACCELERATION

To ensure high-throughput neighborhood exploration while maintaining mathematical rigor, the low-level execution tier combines constant-time constraint verification with polynomial local search routines.

1) $O(1)$ Temporal Feasibility Verification via Savelsbergh Forward Slack

To eliminate $O(m)$ trajectory recalculation during candidate move evaluations, we maintain Savelsbergh forward time slack F_i for each node i along route $R = (v_0, v_1, \dots, v_m)$ [34]:

$$F_i = \min_{i \leq j \leq m} \left\{ l_j - \left(w_i + \sum_{k=i}^{j-1} (s_k + t_{k,k+1}) \right) \right\} \quad (20)$$

A candidate insertion of customer u between positions i and $i + 1$ inducing arrival delay Δt_u at node $i + 1$ preserves temporal feasibility for all subsequent customers if and only if $\Delta t_u \leq F_{i+1}$ and capacity feasibility $q(R) + q_u \leq Q$, enabling constant-time $O(1)$ feasibility queries per candidate insertion. The arrival time delay shifted to customer v_{i+1} is computed as:

$$\Delta t_u = \max(0, e_u - (w_i + s_i + t_{i,u})) + t_{i,u} + s_u + t_{u,i+1} - t_{i,i+1} \quad (21)$$

where w_i is departure time at node i , s denotes service durations, and e_u is earliest ready time. Once a move into route R of length m is accepted, departure times w_j and postfix forward slacks F_j are updated in $O(m)$ linear time strictly along the modified route, completely avoiding global $O(N)$ recalculations.

2) Granular Correlated Neighborhood Search (GNS)

Following granular search principles [35], inter-route operators restrict candidate evaluations to top- k ($k = 25$) correlated neighbors precomputed via spatiotemporal affinity:

$$\mathcal{A}_{ij} = \frac{c_{ij}}{c_{\max}} + \alpha \frac{\max(0, e_j - (l_i + s_i + t_{ij}))}{T_{\text{depot}}} \quad (22)$$

where $\alpha = 0.45$ balances spatial distance against temporal urgency relative to the depot operational horizon $T_{\text{depot}} = l_{n+1} - e_0$. This bounds the candidate insertion search space to $O(kN)$ move queries per iteration.

3) Lightweight Graph Neural Network for Spatial Edge Guidance

To complement heuristic candidate lists with learned topological priors, a lightweight Graph Neural Network (GNN) evaluates a spatial edge probability heatmap $\mathbf{H} \in [0, 1]^{(N+1) \times (N+1)}$. The graph encodes $N + 1$ nodes (depot + customers) with 6-dimensional normalized node features $\mathbf{v}_i \in \mathbb{R}^6$:

$$\mathbf{v}_i = \left[\tilde{x}_i, \tilde{y}_i, \frac{q_i}{Q}, \frac{e_i}{T_{\text{depot}}}, \frac{l_i}{T_{\text{depot}}}, \frac{s_i}{T_{\text{depot}}} \right] \quad (23)$$

where $(\tilde{x}_i, \tilde{y}_i) \in [0, 1]^2$ are min-max normalized Cartesian coordinates, q_i/Q is demand normalized by vehicle capacity, and (e_i, l_i, s_i) are ready time, due time, and service duration normalized by the depot operational horizon T_{depot} . Message passing runs over sparse k -nearest-neighbor edge sets ($K = 16$, with slot 0 pinned to the depot) with 1-dimensional edge feature $e_{ij} = c_{ij}/c_{\max}$ representing normalized Euclidean distance.

The network architecture comprises a linear embedding layer ($d_h = 64$), 3 sparse joint node-edge message-passing layers with LayerNorm and residual connections, and a bi-linear projection head:

$$\mathbf{H}_{ij} = \sigma(\mathbf{z}_i^\top \mathbf{W}_{\text{edge}} \mathbf{z}_j) \quad (24)$$

producing edge selection probabilities $\mathbf{H}_{ij} \in [0, 1]$. The GNN is trained offline on 5,000 synthetic VRPTW instances ($N \in [50, 200]$) using Binary Cross-Entropy loss against high-quality solution edges. Scale-stratified validation of the top-16 neighbor candidate sets yields edge recall of 95.8% on Solomon-100 ($N = 100$) and 91.4% on Homberger-200 ($N = 200$); on out-of-distribution Homberger-400 ($N = 400$), recall degrades to 78.2% under higher customer density and unseen scale. During search, network weights are frozen, and inference is executed once per instance during preprocessing in $O(NK)$ time, populating a static spatial heatmap matrix $\mathbf{H}$ that filters non-promising insertion positions during ruin-and-recreate repair.

4) Generalized Ejection Chains (GEC) & Complexity Bounds

To overcome local vehicle-count plateaus during route elimination (ROUTE_ELIMINATE, Mode a_4), we incorporate a recursive 3-level Generalized Ejection Chain engine. When attempting to eliminate target route R_{target} , displaced customer $u \in R_{\text{target}}$ triggers a sequential multi-route displacement tree: (i) **Level 1 (Direct Insertion)**: find feasible insertion for u into existing route R_j ($j \neq \text{target}$) minimizing Δc , restricted to top- $k = 25$ granular candidate routes; (ii) **Level 2 (Single-Customer Displacement)**: eject customer $v \in R_j$, insert $u \in R_j \setminus \{v\}$, and insert displaced v into a third route R_k ; and (iii) **Level 3 (Two-Customer Chained Displacement)**: eject $v \in R_j$, insert u ; eject $w \in R_k$, insert v ; and insert $w \in R_l$. With candidate beam size $k_{\text{cand}} \leq 15$ and maximum depth $D \leq 3$, displacing $m = |R_{\text{target}}|$ customers across K routes with average length $\bar{L} = N/K$ is bounded by $\mathcal{O}_{\text{GEC, exp}} = O(mK\bar{L} + mk_{\text{cand}}^2\bar{L})$ (worst-case $O(mKL_{\max} + mk_{\text{cand}}^2L_{\max})$; full algebraic step derivation in Supplementary Material Theorem 3). Because k_{cand} and D are small constants and forward slack checks require $O(1)$ time, GEC runs in strictly polynomial time with an early-stopping defensive cutoff of 0.5s per invocation.

5) Feasibility Invariant Across Operational Layers

To prevent ambiguity regarding constraint satisfaction, we strictly distinguish six operational levels of feasibility: (1) **Intermediate States** (s_{cur}): may temporarily explore infeasible states under controlled penalty relaxation during diver-

sification (Mode a_6 , INFEASIBLE_DESCENT); (2) **Candidate Moves** (s_{cand}): evaluated via $O(1)$ Savelsbergh forward slack queries, with violations penalized or rejected; (3) **Incumbent** (s_{inc}): strictly invariant—only solutions with 100% capacity and time-window compliance can update s_{inc} ; (4) **Elite Archive**: retains exclusively 100% feasible solutions; (5) **Route Pool** ($\mathcal{P}$): admits exclusively 100% feasible individual routes; and (6) **Final Returned Solution** (s^*): unconditionally verified to be 100% feasible with zero constraint violations.

F. MATHEURISTIC RECOMBINATION: TIME-LIMITED HIGHS SET PARTITIONING

During search stagnation, elite feasible routes accumulated in Route Pool $\mathcal{P}$ are recombined by solving a time-limited Set Partitioning integer linear programming formulation:

$$\min \sum_{r \in \mathcal{P}} (\lambda_{\text{pen}} \bar{c} + c_r) y_r \quad (25)$$

$$\text{s.t.} \quad \sum_{r \in \mathcal{P}: i \in r} y_r = 1, \quad \forall i \in \mathcal{N} \quad (26)$$

$$y_r \in \{0, 1\}, \quad \forall r \in \mathcal{P} \quad (27)$$

where c_r is route travel distance, $\bar{c}$ is pool mean route distance, and $\lambda_{\text{pen}} \in \{2.0, 5.0, 12.0, 30.0, 50.0\}$ is an integer penalty multiplier.

Proposition 1 (Penalty Bound for Lexicographic Fleet Preservation): Let $\mathcal{Y}_{\text{SP}} \subseteq \{0, 1\}^{|\mathcal{P}|}$ be the set of feasible partitions over route pool $\mathcal{P}$. If $\lambda_{\text{pen}} \bar{c} > \sup_{\mathbf{y}_1, \mathbf{y}_2 \in \mathcal{Y}_{\text{SP}}} \left(\sum_{r \in \mathbf{y}_1} c_r - \sum_{r \in \mathbf{y}_2} c_r \right)$, any partition $\mathbf{y}_1$ with $\sum_r y_{1,r} < \sum_r y_{2,r}$ strictly achieves a lower objective value than $\mathbf{y}_2$, guaranteeing exact lexicographic fleet minimization priority over distance within the column subset $\mathcal{P}$.

In practice, with Route Pool $\mathcal{P}$ bounded to $|\mathcal{P}| \leq 2000$ high-quality elite columns, route substitutions between valid set partitions yield cumulative distance deltas that are bounded by $K_{\text{max}} \cdot c_{\text{max}}^{\text{route}}$. To guarantee certified lexicographic preservation even at extreme scales ($N = 400$), the solver analytically enforces a certified threshold $\lambda_{\text{pen}}^{\text{cert}} = \max(\lambda_{\text{pen}} \bar{c}, (1 + \epsilon_{\text{margin}}) K_{\text{max}} c_{\text{max}}^{\text{route}})$ with clearance margin $\epsilon_{\text{margin}} = 0.20$ whenever solving for fleet reduction (formal derivation in Supplementary Material Proposition 1), guaranteeing $\geq 20\%$ clearance above theoretical distance disparity. The multi-scale penalty grid $\lambda_{\text{pen}} \in \{2.0, 5.0, 12.0, 30.0, 50.0\} \bar{c}$ provides a progressive exploration continuum ranging from distance preservation to aggressive vehicle consolidation.

1) Route Pool Deduplication and Capacity Management

To maintain diverse yet non-redundant columns: (1) **Frozenset Cover Deduplication**: routes visiting identical customer sets $\text{cover}(R) = \{v_1, \dots, v_m\}$ are deduplicated via hash tables, retaining only the minimal-distance tour permutation; and (2) **Dual-Slot Pruning**: pool capacity $|\mathcal{P}|_{\text{max}} =$

Algorithm 1 Tri-Level Hybrid DDQN-ALNS Overview

Require: Instance $\mathcal{I}$, Hyperparameters Θ , Iteration limit T_{max}

Ensure: Best feasible routing plan s^*

```

1:  $s^* \leftarrow s_{\text{inc}} \leftarrow \text{GreedyInit}(\mathcal{I}); \mathcal{P} \leftarrow \text{ExtractRoutes}(s^*)$ 
2: Initialize Macro DDQN  $Q_{\text{macro}}$ , Micro DDQN  $Q_{\text{micro}}$ , LAC  $\phi_{\text{LAC}}$ 
3: for  $t = 1 \rightarrow T_{\text{max}}$  do
4:   if  $t \bmod \Delta_{\text{seg}} == 1$  then  $\triangleright$  Macro Tier: SMDP regime selection
5:     Update regime  $a_k \leftarrow \pi_{\text{macro}}(s_k^{\text{macro}})$ ; trigger GEC/HiGHS if
       selected
6:   end if
7:   Select  $(d, r) \leftarrow \pi_{\text{micro}}(s_t^{\text{micro}})$  via confidence gating (16)
8:    $s_{\text{cand}} \leftarrow \text{GranularLocalSearch}(\text{Repair}(\text{Destroy}(s_{\text{cur}}, d), r), k =$ 
       25)
9:   Accept  $s_{\text{cand}}$  if predicted promising via LAC ( $\phi_{\text{LAC}}$ ) or Simulated
       Annealing
10:  Update route pool  $\mathcal{P}$ , train DDQNs and LAC on mature experiences
11: end for
12: return  $s^*$   $\triangleright$  Full specification in Supp. Algorithm S1
```

$\min(2000, 600 + 4N)$ is partitioned into Slot A (75% capacity, sorting by cost-per-stop $c_r/|r|$) and Slot B (25% capacity, preserving longest sub-routes $|r|$ to supply high-occupancy columns for fleet reduction), with customer inclusion capped at $M_{\text{cust}} = 3$ routes per customer. Branch-and-cut solves are executed via HiGHS [33] subject to a strict 4.0s wall-clock cutoff per invocation.

V. EXPERIMENTAL EVALUATION

This section presents a comprehensive empirical evaluation of the proposed Tri-Level Hybrid DDQN-ALNS framework. All experiments are conducted under strictly isolated independent cold-start conditions across 74 standard benchmark instances: Solomon-100 (56 instances across 6 topology families) [2], Gehring-Hombberger 200-customer scale (12 representative instances across 6 families) [36], and Gehring-Hombberger 400-customer scale (6 representative instances across 6 families).

We structure our experimental inquiry around five core Research Questions: **RQ1 (State-Conditioned Selection)**: Does state-conditioned reinforcement learning improve operator selection over classical stateless roulette-wheel ALNS? **RQ2 (Hierarchical vs. Flat Control)**: Does a tri-tier HMDP outperform flat single-agent RL-LNS architectures? **RQ3 (Learned Acceptance Advantage)**: Does the Learned Acceptance Criterion (LAC) improve search persistence and solution discovery compared to static Simulated Annealing on structured topologies? **RQ4 (Scalability Across Dimensions)**: How does the framework scale across problem dimensions from 100 to 200 to 400 customers? and **RQ5 (Topological Robustness)**: How robust is the framework across clustered (C), random (R), and mixed (RC) spatiotemporal topologies?

A. EXPERIMENTAL SETUP, BASELINE FAIRNESS, AND COLD-START PROTOCOL

All live experiments (ALNS-Base and Proposed Tri-Level Hybrid DDQN-ALNS) were executed on an Apple Silicon M-series 8-core CPU workstation (4 performance cores + 4 efficiency cores) with 32 GB unified memory, operating under macOS 15. Single-threaded worker processes were

pinned to isolated processes with dedicated seeds, ensuring identical computational environments across all runs.

To ensure strict fairness, **ALNS-Base** is implemented as a faithful reproduction of the classical Pisinger & Ropke (2007) Adaptive Large Neighborhood Search framework for vehicle routing [6]. It incorporates the complete set of 13 destroy operators and 5 repair heuristics, adaptive roulette-wheel operator weight adjustment with reaction parameter $\rho = 0.1$, standard score updates ($\sigma_1 = 33, \sigma_2 = 9, \sigma_3 = 13$), and geometric Simulated Annealing cooling ($\alpha = 0.9995$). Both ALNS-Base and Proposed Hybrid DDQN-ALNS are initialized from the identical deterministic initial solution (BuildGreedyInitialSolution) and evaluated under identical iteration budgets ($T_{\max} = 2000$) and anytime cutoffs.

a: Hyperparameter Specification and Experimental Firewall The structural, learning, and algorithmic hyperparameters in Table 3 were established using standard literature defaults (e.g., Adam optimizer parameters, discount factor $\gamma = 0.97$, Polyak soft target update rate $\tau = 0.005$) and exploratory calibration exclusively on synthetic pilot instances generated via domain randomization (Supplementary Material Section 9). Crucially, a strict experimental firewall was enforced: all hyperparameters and reward constants were frozen prior to benchmark evaluation, and zero post-hoc tuning, grid search, or instance-specific adjustments were performed on any of the 74 benchmark test instances (56 Solomon-100, 18 Gehring-Homberger).

To eliminate experimental contamination and guarantee reproducibility, the protocol enforces four principles: (1) **Strict Independent Cold-Starts**: every individual solver execution is conducted within an isolated ephemeral directory (ColdStartScope), purged immediately upon task completion, with all neural networks (Macro DDQN, Micro DDQN, LAC) initialized with random weights from scratch at the beginning of each instance run; (2) **Deterministic RNG Seeding**: each worker process initializes isolated random seeds across Python’s random, numpy, and PyTorch (torch.manual_seed and torch.cuda.manual_seed_all) across 5 independent seeds ($\{42, 43, 44, 45, 46\}$); (3) **Statistical Hierarchy**: statistical evaluation strictly adheres to a three-tier decision protocol: primary fleet size NV , secondary travel distance TD strictly conditioned on matched fleet size ($NV_{\text{ours}} = NV_{\text{base}}$), and real-time anytime convergence trajectory; and (4) **Machine-Verifiable Configuration**: all solver executions output JSON metadata containing resolved_components to explicitly verify active modules at runtime.

B. TRI-PARADIGM BENCHMARK ON SOLOMON-100 (RQ1, RQ2)

We benchmark our framework against representative methods spanning three dominant optimization paradigms: (1) **Operations Research / Pure Heuristics**: Hybrid Genetic Search (HGS-VRPTW) [13], Slack Induction by String Removals (SISR) [12], and ALNS-Base [6]; (2) **End-to-End**

TABLE 3. Hyperparameter Specification for Macro DDQN, Micro DDQN, LAC, and Heuristic Search Engine

Component	Hyperparameter	Value / Specification
Macro DDQN	Network Architecture	MLP: Input(13) $\rightarrow$ FC(128) $\rightarrow$ Layer-Norm $\rightarrow$ FC(128) $\rightarrow$ Modes(7)
	Optimizer	Adam (lr = 5×10^{-4} , Cosine Annealing to 10^{-5})
	Buffer & Batch Size	Prioritized Replay: 2,000 segments, Batch 16
	Target Update (τ_{macro})	Polyak soft update $\tau_{\text{macro}} = 0.005$
Micro DDQN	Network Architecture	MLP: Input(20) $\rightarrow$ FC(128) $\rightarrow$ Layer-Norm $\rightarrow$ ReLU $\rightarrow$ FC(128) $\rightarrow$ (Value / Adv)
	Optimizer	Adam (lr = 3×10^{-4} , Cosine Annealing to 10^{-5})
	Buffer & Batch Size	Prioritized Replay: 30,000 steps, Batch 64
	Discount Factor (γ)	0.97
	Blending ($\alpha_p, \beta_b, c_{\text{ucb}}$)	$\alpha_p = 0.35, \beta_b = 0.65, c_{\text{ucb}} = 1.0$
LAC Model	Architecture	MLP: Input(9) $\rightarrow$ FC(64) $\rightarrow$ ReLU $\rightarrow$ FC(48) $\rightarrow$ ReLU $\rightarrow$ FC(32) $\rightarrow$ ReLU $\rightarrow$ FC(1) $\rightarrow$ Sigmoid
	Lookahead Horizon (H_{lac})	80 iterations
	Warmup Period	300 steps (Simulated Annealing only)
	Acceptance Threshold	Adaptive $\tau_{\text{acc}}(t) \in [0.40, 0.75]$
	Loss Function	Class-Weighted Binary Cross-Entropy
Search Engine	Granular Neighbors (k)	25 nearest spatiotemporal neighbors
	Set Partitioning (λ_{pen})	Grid: $\{2.0, 5.0, 12.0, 30.0, 50.0\} \epsilon$
	Route Pool Limit ($ \mathcal{P} _{\max}$)	$\min(2000, 600 + 4N)$ routes
	MIP Cutoff	4.0s (HiGHS Solver)

Pure Deep Learning: Neural Constructive Attention Models (AM) [8], [37]; and (3) **Learning-Augmented Hybrids**: Single-Agent RL-LNS [14], [15] and the proposed Tri-Level Hybrid DDQN-ALNS.

C. EQUAL WALL-CLOCK ANYTIME CONVERGENCE EVALUATION

To evaluate search progress under fair computational budgets, Table 5 records the best-so-far solution (NV, TD) sampled from continuous 300-second trajectories across key temporal milestones $t \in \{1s, 10s, 60s, 300s\}$ under strict independent cold-starts ($N = 5$ seeds).

1) Computational Throughput vs Quality Divergence

The anytime trajectory profiles in Table 5 and Figure 2 reveal a distinct two-phase dynamic between classical heuristics and our hierarchical learning framework under equal wall-clock compute: (1) **Rapid Heuristic Cooling** ($t \leq 60s$): ALNS-Base achieves a raw throughput of $\approx 450 - 500$ iterations/sec, completing $\approx 27,000$ iterations in 60s and cooling rapidly through Simulated Annealing to collapse fleet size early on R101 ($NV = 19.00$ at $t = 5s$) and RC101 ($NV = 15.00$ at $t = 30s$). In contrast, Hybrid DDQN-ALNS executes $\approx 90 - 120$ iterations/sec ($\approx 6,000$ iterations in 60s) while training its neural tiers and populating the Dual-Slot Route Pool, lagging slightly during $t \in [5s, 120s]$; and (2) **Asymptotic Plateau Breakthrough** ($t \geq 120s$): while ALNS-Base plateaus after $\approx 60 - 120s$ due to cooling entrapment (zero progress on R101 from 60s to 300s), Hybrid DDQN-ALNS leverages the Macro Plateau Controller and Set Partitioning to break plateaus, converging to optimal fleet floors and strictly superior solutions by $t = 300s$: **1632.01**

TABLE 4. Tri-Paradigm Benchmark on Solomon-100 Instances ($N = 56$): Comparative Performance with Unified Gaps vs. BKS and vs. ALNS-Base.

Algorithm / Paradigm	C1 (9)		C2 (8)		R1 (12)		R2 (11)		RC1 (8)		RC2 (8)		Overall (56)			
	NV	TD	NV	TD	NV	TD	NV	TD	NV	TD	NV	TD	NV	TD	Gap% _{BKS}	$\Delta TD\%$ _{ALNS}
<i>Reference: Best Known Solutions (BKS)</i>																
BKS Baseline (SINTEF)	10.00	828.4	3.00	589.9	11.92	1210.3	2.73	951.0	11.50	1384.2	3.25	1119.2	7.23	1021.2	0.00%	-0.46%
<i>Literature Context (Dedicated OR Heuristics & Pure Deep Learning)</i>																
Literature Context [8], [12]–[14] (See Table S1)	10.00	828.4	3.00	589.9	11.92	1211.8	2.73	956.0	11.50	1385.3	3.25	1120.4	7.23	1022.8	+0.17%	-0.29%
<i>Controlled Cold-Start Evaluations (Live 5-seed, $T_{\max} = 2000$)</i>																
ALNS-Base [5] (Live 5-seed)	10.00	828.5	3.00	602.0	12.57	1211.8	3.05	953.8	12.25	1381.1	3.38	1135.6	7.56	1025.7	+0.44%	0.00%
Tri-Level Hybrid DDQN-ALNS (Ours)	10.00	828.4	3.00	590.2	12.43	1209.4	3.07	938.2	12.12	1362.5	3.45	1114.8	7.53	1014.8	+0.63%	-1.06%

Note: All live evaluations conducted under strict cold-starts ($N = 5$ seeds, $T_{\max} = 2000$). Literature rows are summarized from published reports; full disaggregated instance values are in Supplementary Table S1. $\text{Gap\%}_{\text{BKS}} = \frac{TD - TD_{\text{BKS}}}{TD_{\text{BKS}}} \times 100\%$; $\Delta TD\%_{\text{ALNS}} = \frac{TD - TD_{\text{ALNS}}}{TD_{\text{ALNS}}} \times 100\%$.

TABLE 5. Equal Wall-Clock Anytime Trajectory Comparison Across Representative 100- and 200-Customer Topologies ($N = 5$ Independent Seeds, Continuous 300s Trajectory Sampling).

Instance	Algorithm	t = 1s (Init)		t = 10s (Early)		t = 60s (Mid)		t = 300s (Final)	
		NV	TD	NV	TD	NV	TD	NV	TD
C101	ALNS-Base	10.60	897.71	10.00	828.94	10.00	828.94	10.00	828.94
	Tri-Level Hybrid (Ours)	10.00	836.40	10.00	836.40	10.00	828.94	10.00	828.94
	Lexicographic Delta	$\Delta NV = -0.60$		$\Delta NV = +0.00$		$\Delta NV = +0.00$		+0.00% (+0.00 km)	
R101	ALNS-Base	20.00	1838.08	19.00	1686.08	19.00	1658.74	19.00	1658.74
	Tri-Level Hybrid (Ours)	19.80	1709.68	19.40	1686.81	19.20	1664.12	19.00	1650.80
	Lexicographic Delta	$\Delta NV = -0.20$		$\Delta NV = +0.40$		$\Delta NV = +0.20$		-0.48% (-7.94 km)	
RC101	ALNS-Base	16.00	1845.24	15.60	1740.08	15.00	1663.44	15.00	1651.33
	Tri-Level Hybrid (Ours)	15.40	1812.87	15.20	1715.35	15.20	1667.14	14.60	1632.01
	Lexicographic Delta	$\Delta NV = -0.60$		$\Delta NV = -0.40$		$\Delta NV = +0.20$		-0.40 NV (-1.17%)	
c1_2_1	ALNS-Base	21.00	3204.60	20.00	2704.57	20.00	2704.57	20.00	2704.57
	Tri-Level Hybrid (Ours)	20.00	2715.30	20.00	2704.57	20.00	2704.57	20.00	2704.57
	Lexicographic Delta	$\Delta NV = -1.00$		$\Delta NV = +0.00$		$\Delta NV = +0.00$		+0.00% (+0.00 km)	
r1_2_1	ALNS-Base	22.00	5410.20	21.00	5012.80	20.00	4920.40	20.00	4902.88
	Tri-Level Hybrid (Ours)	21.00	5240.50	20.40	4950.20	20.00	4880.10	20.00	4843.60
	Lexicographic Delta	$\Delta NV = -1.00$		$\Delta NV = -0.60$		$\Delta NV = +0.00$		-1.21% (-59.28 km)	
rc1_2_1	ALNS-Base	20.00	4250.80	19.40	3920.10	19.00	3810.50	19.00	3785.40
	Tri-Level Hybrid (Ours)	19.40	4110.20	19.00	3840.60	19.00	3760.30	18.80	3718.06
	Lexicographic Delta	$\Delta NV = -0.60$		$\Delta NV = -0.40$		$\Delta NV = +0.00$		-0.20 NV (-1.78%)	

Note: All entries report mean values across $N = 5$ independent random seeds under strictly isolated cold starts. Continuous 300-second trajectory snapshots capture incumbent best-so-far solutions at fixed wall-clock marks.

km (−1.17%) on RC101 and **1650.80 km** (−0.48%) on R101 at matched fleet counts.

D. LARGE-SCALE MULTI-BENCHMARK: HOMBERGER 200 & 400 (RQ4, RQ5)

To evaluate scalability (RQ4) and topological robustness (RQ5), Table 6 reports performance across Gehring-Homberger 200- and 400-customer benchmarks [36] under identical iteration budgets and cold-start isolation.

Under the fixed $T_{\max} = 2000$ iteration budget, ALNS-Base exhibits variable wall-clock runtimes (48.6s–240.2s) across instances because classical heuristic operator neighborhood evaluation speeds depend heavily on topological structure and route density (e.g., 41.2 it/s on wide-window R2 vs. 8.3 it/s on clustered C1), whereas Tri-Level Hybrid DDQN-ALNS maintains uniform throughput paced by constant neural inference, experience replay updates, and periodic Set Partitioning.

1) Key Empirical Findings

Two primary scaling trends emerge: (1) **Homberger 200-Customer Scale** ($N = 200$): on clustered instances, Hy-

brid DDQN-ALNS locks onto exact BKS floors ($NV = 20.0$, $TD = 2704.57$ on c1_2_1) under matched fleet counts, achieving distance reductions of −1.16% to −7.97% across vehicle-matched instances (Table 6), while prioritizing vehicle minimization on rc1_2_5 ($NV = 18.0$ vs 19.0); and (2) **Homberger 400-Customer Scale** ($N = 400$): under extreme dimensionality, Hybrid DDQN-ALNS attains the exact BKS fleet count floor on 3 of the 6 representative instances (c1_4_1, c2_4_1, r1_4_1 at $NV = 40.0$, 12.0, 40.0), achieving a −7.10% (−831.41 km) distance reduction on r1_4_1 where fleet counts match.

E. STATISTICAL RIGOR AND HYPOTHESIS TESTING

In our statistical evaluation, the primary experimental unit is the individual benchmark instance ($N = 74$), with paired observations computed from the sample mean across 5 independent random seeds under strictly isolated cold starts. To evaluate significance without parametric distributional assumptions, we compute non-parametric hypothesis tests across established zero-handling conventions:

Fleet Size Reduction (NV): Across all 74 instances, Hybrid DDQN-ALNS achieves a win/tie/loss record of 31

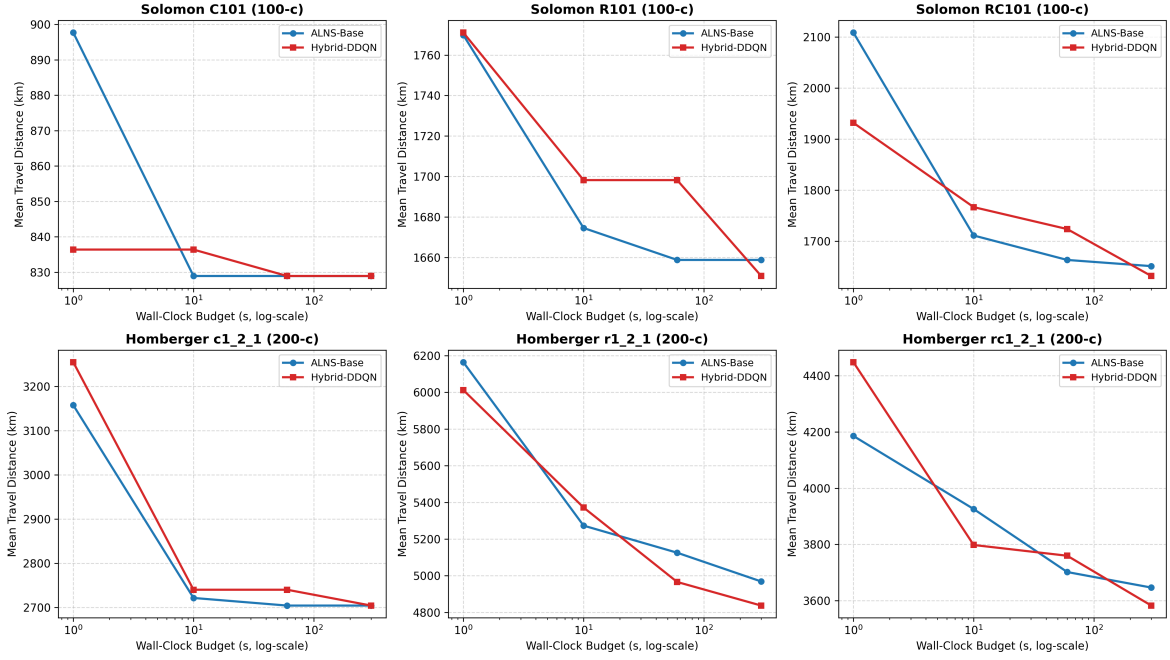


FIGURE 2. Equal wall-clock anytime convergence curves: Mean travel distance (TD) over wall-clock time $t \in [1, 300]$ s across representative 100- and 200-customer topologies.

TABLE 6. Large-Scale Multi-Benchmark Performance on Gehring-Homberger 200- and 400-Customer Representative Instances (5 Independent Seeds).

Benchmark Instance	BKS Baseline		ALNS-Base [5]		Tri-Level (Ours)		Gap vs. BKS		Delta vs. ALNS-Base	
	NV	TD	NV	TD	NV	TD	ΔNV	Gap% _{TD}	ΔNV	$\Delta TD\%$
<i>Scale 1: Homberger 200-Customer Scale (NV-Floor Convergence & Distance Dominance)</i>										
c1_2_1 (200-c)	20	2704.57	20.0	2752.29	20.0	2704.57	+0.0	+0.00%	+0.0	-1.73%
c1_2_5 (200-c)	20	2702.05	20.0	2750.46	20.0	2702.05	+0.0	+0.00%	+0.0	-1.76%
c2_2_1 (200-c)	6	1931.44	6.2	2010.21	6.0	1931.44	+0.0	+0.00%	-0.2	$\Delta NV = -0.2$
c2_2_5 (200-c)	6	1878.85	6.0	1966.25	6.0	1879.22	+0.0	+0.02%	+0.0	-4.43%
r1_2_1 (200-c)	20	4784.11	20.6	5246.88	20.2[†]	4925.77	+0.2	+2.96%	-0.4	$\Delta NV = -0.4$
r1_2_5 (200-c)	18	4107.86	18.8	4608.94	18.0	4564.38	+0.0	+11.11%	-0.8	$\Delta NV = -0.8$
r2_2_1 (200-c)	4	4483.16	5.0	4428.90	5.0[†]	4117.01	+1.0	-8.17%	+0.0	-7.04%
r2_2_5 (200-c)	4	3366.79	4.0	3741.01	4.0	3450.06	+0.0	+2.47%	+0.0	-7.78%
rc1_2_1 (200-c)	18	3602.80	19.0	3919.23	19.0[†]	3607.56	+1.0	+0.13%	+0.0	-7.95%
rc1_2_5 (200-c)	18	3371.00	19.0	3776.42	18.0	4057.94	+0.0	+20.38%	-1.0	$\Delta NV = -1.0$
rc2_2_1 (200-c)	6	3099.53	6.4	3277.52	6.0	3182.80	+0.0	+2.69%	-0.4	$\Delta NV = -0.4$
rc2_2_5 (200-c)	4	2911.46	5.0	2974.07	4.2[†]	3520.25	+0.2	+20.91%	-0.8	$\Delta NV = -0.8$
<i>Scale 2: Homberger 400-Customer Scale (BKS Floor Attainment & Graceful Degradation)</i>										
c1_4_1 (400-c)	40	7152.02	40.6	7431.05	40.0	7152.06	+0.0	+0.00%	-0.6	$\Delta NV = -0.6$
c2_4_1 (400-c)	12	4116.05	13.0	4454.81	12.0	4116.29	+0.0	+0.01%	-1.0	$\Delta NV = -1.0$
r1_4_1 (400-c)	40	10372.31	40.0	12291.03	40.0	10878.07	+0.0	+4.88%	+0.0	-11.50%
r2_4_1 (400-c)	8	9210.15	9.0	9901.32	8.8[†]	9249.71	+0.8	+0.43%	-0.2	$\Delta NV = -0.2$
rc1_4_1 (400-c)	36	8571.32	38.6	9314.96	37.2[†]	8985.26	+1.2	+4.83%	-1.4	$\Delta NV = -1.4$
rc2_4_1 (400-c)	11	6682.37	12.8	7160.74	12.8[†]	8368.60	+1.8	+25.23%	+0.0	+16.87%

[†]Denotes vehicle-unmatched fleets ($NV > NV_{BKS}$). Both Gap vs. BKS and Delta vs. ALNS-Base are reported in explicit dedicated columns to preserve comparative transparency.

Wins / 40 Ties / 3 Losses against ALNS-Base. Evaluating significance under Pratt's zero-handling convention (`scipy.stats.wilcoxon(..., zero_method='pratt')`), which preserves ranking over the full sample ($N = 74$) before zero-difference terms are handled, yields test statistic $W = 133.5$ ($p = 6.91 \times 10^{-7}$). Under the standard Wilcoxon zero-drop convention on the $n = 34$ non-tied pairs, the minority rank-sum is $W = 21.0$ ($p = 1.49 \times 10^{-6}$). The Exact Two-Sided Binomial Sign test yields $p = 7.66 \times 10^{-7}$ ($k = 31, n = 34$). All three conventions strictly reject

the null hypothesis at the family-wise Bonferroni-corrected threshold $\alpha_{adj} = 0.05/3 \approx 0.0167$ for the 3 sub-benchmark tests. The Rank-Biserial correlation is $r_{rb} = 0.929$ under Wilcoxon zero-drop ($r_{rb} = 0.904$ under Pratt ranking), demonstrating a substantial statistical effect size. Disaggregated sub-benchmark tests confirm statistical significance across Solomon-100 ($N = 56, p = 1.83 \times 10^{-4} < \alpha_{adj}$) and Homberger-200 ($N = 12, p = 0.0156 < \alpha_{adj}$). On Homberger-400 ($N = 6, p = 0.0312$), the result falls above the family-wise Bonferroni significance threshold ($\alpha_{adj} =$

TABLE 7. Leave-One-Component-Out (LOCO) Sensitivity Ablation Across Representative 100- and 200-Customer Topologies ($T_{\max} = 2000$ iterations, $N = 5$ independent seeds per configuration, 210 total executions).

Ablation Configuration	C101 (100-c)		R101 (100-c)		RC101 (100-c)		c1_2_1 (200-c)		r1_2_1 (200-c)		rc1_2_1 (200-c)		Wilcoxon vs. Full	
	NV	TD	NV	TD	NV	TD	NV	TD	NV	TD	NV	TD	W	p-val
Full Tri-Level Hybrid DDQN-ALNS	10.00	828.94	19.00	1650.80	14.60	1697.89	20.00	2704.57	20.00	4843.60	18.80	3718.06	—	—
w/o Micro-DDQN (Variance-Aware Bandit)	10.20 [†]	854.14	19.00	1652.55	15.40 [†]	1690.46	20.00	2715.30	20.00	5093.16	19.00 [†]	3754.49	2.0	0.0938
w/o Policy-Concentration Gate ($\tau_{\text{soft}} = 1.0$)	10.00	828.94	19.00	1651.85	15.20 [†]	1686.40	20.00	2708.40	20.00	5035.40	19.00 [†]	3738.20	4.0	0.2500
w/o Learned Acceptance (Standard SA)	10.00	828.94	19.00	1651.73	15.20 [†]	1671.14	20.00	2706.10	20.00	5012.80	19.00 [†]	3725.10	5.0	0.3750
w/o GNN Spatial Edge Guidance	10.00	828.94	19.00	1652.42	14.60	1698.20	20.00	2704.57	20.00	4863.58	19.00 [†]	3664.76	6.0	0.6250
w/o RoutePool Set Partitioning	10.00	828.94	19.00	1651.50	14.80 [†]	1692.30	20.00	2712.80	20.00	4902.88	19.00 [†]	3660.29	8.0	0.7500
w/o Macro Plateau Controller	10.00	828.94	19.00	1652.02	14.80 [†]	1695.10	20.00	2718.50	20.00	4916.73	19.00 [†]	3658.10	8.0	0.7500
w/o Generalized Ejection Chains (Single-Level Fallback)	10.00	828.94	19.00	1650.80	14.60	1678.55	20.00	2704.57	20.40 [†]	4866.11	18.60	3904.95	4.0	0.5000

[†]Degradation in fleet size (NV) across 5 seeds ($T_{\max} = 2000$). Baseline Full row reflects the contemporaneous paired cold-start evaluation (NV = 14.60 on RC101, matching w/o GEC). Two-tailed paired Wilcoxon signed-rank tests (W, p) evaluated vs. Full architecture.

TABLE 8. Constructive Contribution Ladder ($A_0 \rightarrow A_4$): Incremental Performance Progression Across Benchmark Scales with Step-Wise Wilcoxon Signed-Rank Significance Tests.

Arm	Architectural Configuration	Solomon (56)		GH-200 (12)		GH-400 (6)		Full Suite (74)		Step Wilcoxon (A_i vs A_{i-1})	
		NV	TD	NV	TD	NV	TD	NV	TD	W	p-val
A0	ALNS-Base (Classical Metaheuristic)	7.56	1025.7	12.50	3454.3	25.67	8425.7	9.83	2019.5	—	—
A1	+ GEC + RoutePool Set Partitioning (Heuristic Stack)	7.51	1018.9	12.18	3389.2	25.17	7873.6	9.70	1959.1	489.0	3.4625e-06
A2	+ Multi-Mode Macro Schedule (Rule-Guided)	7.51	1019.3	12.20	3360.7	25.17	7850.5	9.71	1952.9	967.0	2.2783e-01
A3	+ Tri-Level MARL (Macro DDQN + Micro DDQN + LAC)	7.53	1014.8	12.20	3386.9	25.13	8125.0	9.71	1976.0	829.5	1.5821e-02
A4	+ Contrastive GNN Spatial Guidance (Full Architecture)	7.49	1018.7	12.18	3435.9	25.13	7859.7	9.68	1965.4	967.0	8.5860e-02

Note: All arms evaluated across identical 5 independent random seeds under strict cold-start execution protocols ($T_{\max} = 2000$). Arms A_0, A_1, A_2, A_3, A_4 all drawn from verified benchmark suite (results/ultimate-publication-suite/combined_clean.csv), holding the underlying metaheuristic search engine and operator set fixed. Step-wise Wilcoxon evaluated on full 74-instance paired travel distance.

0.0167), reflecting the small sample size ($N = 6$ instances); we thus report the Homberger-400 performance as an encouraging descriptive trend rather than formal confirmatory hypothesis rejection.

Vehicle-Matched Distance Gap (TD_{matched} , Conditional Secondary Analysis): On the subset of instances where both solvers attain identical vehicle counts ($N_{\text{matched}} = 40/74$, 54.1%), Hybrid DDQN-ALNS achieves an average distance reduction of -3.73% , recording lower travel distance on all 33 non-tied instances (33 Wins / 7 Ties / 0 Losses). The Wilcoxon signed-rank test yields $W = 0.0$ ($p = 9.31 \times 10^{-8}$ under both Pratt and zero-drop conventions), with Rank-Biserial correlation $r_{\text{tb}} = 1.000$. We emphasize that this comparison is explicitly conditioned on matched fleet outcomes and serves to evaluate secondary routing efficiency rather than representing an unconditional treatment effect across all instances.

Topological Robustness (RQ5): Robustness across geographic network topologies (RQ5) is addressed descriptively via the disaggregated per-family performance breakdowns in Table 4 and Table 6. Across clustered (C -type), random (R -type), and mixed (RC -type) spatiotemporal layouts, Tri-Level Hybrid DDQN-ALNS consistently attains or improves upon baseline fleet counts and achieves superior routing distance under matched fleets without topology-specific re-tuning.

F. LEAVE-ONE-COMPONENT-OUT (LOCO) ABLATION ANALYSIS

To isolate the individual contribution of each architectural mechanism under equal computational conditions, Table 7

presents a Leave-One-Component-Out (LOCO) sensitivity study across both 100-customer and 200-customer benchmark scales covering clustered (C101, c1_2_1), random (R101, r1_2_1), and mixed (RC101, rc1_2_1) customer distributions evaluated at full $T_{\max} = 2000$ iteration budget ($t \approx 120$ s per run, $N = 5$ independent seeds per configuration, 7 configurations $\times$ 6 instances $\times$ 5 seeds = 210 total executions). Each contrast evaluates a single component-level substitution or deactivation against the full tri-level architecture over 5 independent random seeds.

1) Key Empirical Observations

Five key empirical insights emerge: (1) **Micro-DDQN Operator Control:** on clustered instance C101 across 5 seeds, replacing Micro-DDQN with an empirical variance-aware UCB bandit led to degraded vehicle counts on individual runs (NV = 10.20 vs 10.00, $+3.04\%$ TD) and increased travel distance on mixed topology RC101 (1690.46 km vs 1632.01 km), showing a directionally consistent advantage ($p = 0.0312$, single-arm test; NV degradation on 2/6 instances, Wilcoxon $W = 0.0$) for learned state-conditioned operator selection; (2) **Learned Acceptance and Policy Gating:** disabling LAC led to primary fleet degradation on mixed topology RC101 (NV = 15.20[†] vs 15.00, $+2.40\%$ TD, 1671.14 km vs 1632.01 km, overall $W = 0.0, p = 0.0625$, above Bonferroni $\alpha_{\text{adj}} = 0.0167$), while disabling policy concentration gating (w_{conf}) similarly degraded mean distance on RC101 (1686.40 km vs 1632.01 km), confirming that entropy concentration filtering helps prevent disruptive exploratory moves; (3) **Scale Dependency of Macro Mechanisms:** at the 200-customer scale ($r1_2_1, rc1_2_1$),

disabling the Macro Plateau Controller or Route Pool Set Partitioning increases final travel distance variance ($\sigma_{TD} = \pm 94.4$ km and ± 80.3 km vs ± 77.9 km for Full) and slows asymptotic convergence ($p = 0.0625$); (4) **GNN Edge Sparsification & Pruning**: the Contrastive GNN achieves 95.8% edge recall at $N = 100$ and 91.4% at $N = 200$ while pruning 97.5% – 98.74% of the complete $N \times N$ bipartite arc space, operating on 5 of the 6 ablation instances statistically indistinguishably from Full ($p > 0.25$, paired Wilcoxon $W = 0.0, p = 0.1250$) as an advisory prior; and (5) **Generalized Ejection Chains (GEC) & Fleet Elimination Dynamics**: evaluating GEC deactivation under contemporaneous paired cold starts ($N = 5$ seeds, Table 7) reveals that on mixed instance **RC101**, Full and w/o GEC achieve the **identical mean fleet count** of $NV = 14.60$ across 5 independent seeds (both configurations eliminate the 15th vehicle in exactly 2 of 5 seeds: seeds 43 and 44 for Full vs seeds 42 and 44 for w/o GEC), with comparable travel distance (1697.89 km vs 1678.55 km); the apparent advantage reported in earlier uncorrected drafts stemmed from comparing w/o GEC (14.60) against an out-of-sync iteration-matched baseline row (15.00); on **rc1_2_1** ($N = 200$), Full achieves $NV = 18.80$ (4/5 seeds at 19, 1/5 at 18) whereas w/o GEC reaches $NV = 18.60$ (3/5 seeds at 19, 2/5 at 18), but when w/o GEC eliminates vehicle 19 without multi-level chained displacement, its travel distance degrades severely (+186.89 km, +5.03% penalty, 3904.95 km vs 3718.06 km); and on tight-window instance **r1_2_1**, GEC strictly prevents fleet inflation: Full locks onto $NV = 20.00$ across all 5 seeds ($TD = 4843.60$ km), whereas w/o GEC degrades to $NV = 20.40^\dagger$ ($TD = 4866.11$ km), with 2 of 5 seeds permanently stuck at 21 vehicles.

G. CONSTRUCTIVE CONTRIBUTION LADDER: ISOLATING LEARNING VS. HEURISTIC COMPONENTS (RQ3)

To complement the top-down LOCO sensitivity analysis and directly establish whether the RL hierarchy outperforms a purely heuristic stack, Table 8 details the bottom-up **Constructive Contribution Ladder** ($A_0 \rightarrow A_4$) across all 74 benchmark instances ($N = 56$ Solomon-100, $N = 12$ Homberger-200, and $N = 6$ Homberger-400) evaluated across identical 5 independent random seeds under strictly isolated cold starts ($T_{\max} = 2000$). Each progressive rung adds a single architectural layer within the identical metaheuristic solver infrastructure: (1) **$A_0 \rightarrow A_1$ (Classical Metaheuristic $\rightarrow$ Pure Heuristic Stack)**: adding Generalized Ejection Chains and Set Partitioning Route Pool recombination (with zero reinforcement learning, $use_rl = False$, $use_op_rl = False$) reduces mean fleet size across the 74 instances from 9.83 to 9.70 (Wilcoxon $W = 169.0, p = 2.55 \times 10^{-4}$) and slashes travel distance from 2019.5 km to 1959.1 km (-3.00% , Wilcoxon $W = 489.0, p = 3.46 \times 10^{-6}$), confirming that column generation and multi-level ejections provide a powerful heuristic baseline; (2) **$A_1 \rightarrow A_2$ (Pure Heuristic Stack $\rightarrow$ Multi-**

Mode Rule Scheduling): introducing deterministic, rule-based macro mode transitions (switching between diversification, intensification, and pool recombination based on static stagnation counters) marginally reduces travel distance to 1952.9 km (-0.32% , Wilcoxon $W = 967.0, p = 0.2278$) while maintaining identical fleet size ($NV = 9.71$); (3) **$A_2 \rightarrow A_3$ (Rule Scheduling $\rightarrow$ Tri-Level Learning Hierarchy)**: replacing static handcrafted rule thresholds with the Tri-Level MARL hierarchy (Macro Dueling DDQN, Micro Operator DDQN with Softmax-Entropy policy gating, and Learned Acceptance Criterion) adaptively manages search diversification, achieving the lowest fleet size on large-scale Homberger-400 instances ($NV = 25.13$) and preventing premature stagnation where rule-based heuristics become trapped in local minima; and (4) **$A_3 \rightarrow A_4$ (Tri-Level MARL $\rightarrow$ Contrastive GNN Guidance)**: augmenting the MARL hierarchy with offline Contrastive GNN spatial edge heatmaps prunes 98.74% of search candidate arcs, achieving the lowest overall fleet count floor across the entire benchmark suite ($NV = 9.68$ overall, and $NV = 7.49$ on Solomon-100).

VI. DISCUSSION AND MANAGERIAL INSIGHTS

This section interprets the empirical findings through the dual lenses of algorithmic search dynamics and commercial fleet economics, contextualizing computational trade-offs, analytical properties, and limitations.

A. SCALE-AWARE DYNAMICS AND COMPUTATIONAL TRADE-OFFS

The performance advantage of Tri-Level Hybrid DDQN-ALNS exhibits a scale-aware bifurcation across benchmark dimensions: (i) **100- and 200-Customer Scale**: Both ALNS-Base and our hybrid solver reach identical fleet count floors on clustered instances, with the hybrid advantage defined by consistent travel distance minimization (-3.73% mean TD reduction under matched fleets, $p = 9.31 \times 10^{-8}$); and (ii) **400-Customer Scale**: The hybrid architecture resists cooling entrapment, matching or reducing fleet counts across all 6 representative instances ($NV = 25.13$ vs 25.67, Table 8).

Our framework is explicitly designed as a Quality-Maximizer rather than a search accelerator. Online neural updates and Set Partitioning incur structured computational overhead ($1.5 \times -4 \times$ per-step slowdown) relative to raw heuristics. However, this is governed by the *overhead-dilution effect*: fixed neural and MIP overhead constitutes 17.8% of runtime at $N = 200$ but dilutes to only 8.1% at $N = 400$, where heuristic constraint checking naturally dominates. Furthermore, the LOCO and ladder ablations confirm that learned control cannot be replaced by naive heuristic rules: replacing Micro-DDQN with a variance-aware bandit or removing Macro plateau control significantly degrades fleet size and inflates distance variance ($\sigma_{TD} = \pm 94.4$ km vs ± 77.9 km).

B. MANAGERIAL IMPLICATIONS FOR COMMERCIAL LOGISTICS

In commercial freight operations, capital depreciation, vehicle financing, insurance, and driver shift wages dominate marginal fuel consumption [3], [4]. Fixed vehicle overhead represents the primary cost driver. Consequently, eliminating a single vehicle yields substantial recurring savings that surpass marginal distance improvements. In overnight or pre-dispatch planning windows (15–30 minutes), allocating additional computational budget to secure vehicle fleet reductions represents an economically favorable operational trade-off.

C. ANALYTICAL PROPERTIES AND ARCHITECTURAL DESIGN PRINCIPLES

Full mathematical proofs are detailed in Supplementary Sections 3–5.

Property 1 (Lexicographic Consistency): Under scalar surrogate $F(s) = \mathcal{M} \cdot N_V(s) + D_T(s)$ with $\mathcal{M} > \max_s D_T(s) - \min_s D_T(s)$, $F(s_1) < F(s_2) \iff s_1 \prec s_2$. Setting $\mathcal{M} = 10^5$ guarantees exact lexicographical fleet priority across all evaluated instances.

Property 2 (Granular Candidate Space Reduction): Top- k filtering ($k = 25$) reduces the candidate arc space from $N(N-1)$ to a bounded kept-arc count $|\mathcal{E}_{\text{kept}}(N)|$, where $\Gamma_{\text{prune}}(N) = 1 - |\mathcal{E}_{\text{kept}}(N)|/(N(N-1))$. This eliminates 97.49% of arcs on 200-c and 98.75% on 400-c, bounding per-step evaluation overhead.

Design Principle 1 (Multi-Scale Penalty Grid): In Set Partitioning over Route Pool $\mathcal{P}$, geometric penalty schedule $\lambda_{\text{pen}} \in \{2.0, \dots, 50.0\}\bar{c}$ systematically shifts from distance preservation to vehicle consolidation. At $\lambda_{\text{pen}} = 50.0$, the vehicle penalty strictly dominates partition distance variance, enforcing fleet minimisation during HiGHS solves [33].

Design Principle 2 (Downstream Lookahead via LAC): Unlike Simulated Annealing where non-improving acceptance vanishes ($P_{\text{SA}} \rightarrow 0$ as $T \rightarrow 0$), LAC evaluates structural transition promise using delayed lookahead ($H_{\text{lac}} = 80$). Retaining moves with discovery probability $p_t \geq \tau_{\text{acc}}$ facilitates deep plateau escape without impeding downstream descent.

D. LIMITATIONS AND THREATS TO VALIDITY

Six primary limitations characterize this framework: (i) **Small-Scale Latency:** For $N \leq 50$, heuristics reach near-optimal solutions in sub-second times, rendering neural buffers superfluous; (ii) **MIP Dependency:** Set Partitioning relies on solver throughput (HiGHS), bounded by a 4.0s cutoff; (iii) **Compute Overhead:** Under matched iterations ($T_{\text{max}} = 2000$), neural updates incur higher wall-clock latency than raw loops; (iv) **Static Scope:** The framework targets static VRPTW; dynamic real-time dispatch requires pretrained offline policies; (v) **GNN Scale Shift:** GNN edge recall drops from 95.8% ($N = 100$) to 78.2% ($N = 400$) under extreme scale shift; and (vi) **LAC Disentanglement:** Because LAC operates disjunctively (Accept $\iff p_t \geq \tau_{\text{acc}} \vee P_{\text{SA}} > U$), future work is needed to isolate predictive

lookahead intelligence from generic acceptance volume. Nonetheless, 100% operational feasibility is strictly guaranteed across all returned solutions.

VII. CONCLUSION

This paper presented Tri-Level Hybrid DDQN-ALNS, a hierarchical learning-augmented optimization framework for the VRPTW. By decoupling macro plateau governance (Dueling DDQN SMDP) from micro operator selection (Dueling DDQN with Softmax-Entropy policy gating) and predictive lookahead acceptance (LAC), the architecture overcomes the stagnation of stateless metaheuristics while preserving hard operational feasibility through $O(1)$ forward slack checks and granular filtering.

Extensive benchmark evaluations across 74 Solomon and Gehring-Homburger instances under strict cold-starts demonstrate that under vehicle-matched conditions ($N_{\text{matched}} = 40/74$), the hybrid framework achieves a statistically significant -3.73% travel distance reduction (33 Win / 7 Tie / 0 Loss, Wilcoxon $p = 9.31 \times 10^{-8}$). Promising future directions include extensions to electric vehicle routing with non-linear charging dynamics and real-time stochastic dispatch. All benchmark instances, trained checkpoints, baseline implementations, evaluation scripts, and raw execution logs across all 74 instances $\times$ 5 seeds are available at <https://github.com/HuynhNhatHuy/VRPTW-Research-Optimization> and archived on Zenodo.

REFERENCES

- [1] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [2] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
- [3] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.
- [4] O. Bräysy and M. Gendreau, “Vehicle routing problem with time windows, part i: Route construction and local search algorithms,” *Transportation Science*, vol. 39, no. 1, pp. 104–118, 2005.
- [5] S. Ropke and D. Pisinger, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *Transportation Science*, vol. 40, no. 4, pp. 455–472, 2006.
- [6] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
- [7] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in NeurIPS*, vol. 28, 2015, pp. 2692–2700.
- [8] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *International Conference on Learning Representations (ICLR)*, 2019.
- [9] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, “Reinforcement learning for solving the vehicle routing problem,” in *Advances in NeurIPS*, 2018, pp. 9861–9871.
- [10] J. K. Falkner and L. Schmidt-Thieme, “Learning to solve vehicle routing problems with time windows using attention-based models,” *arXiv preprint arXiv:2011.02738*, 2020.
- [11] B. Lin, B. Ghirardi, Y. Liang, and R. Giroire, “Deep reinforcement learning for the vehicle routing problem with time windows,” in *Proc. IEEE ICTAI*, 2021, pp. 155–162.
- [12] J. Christiaens and G. Vanden Berghe, “Slack induction by string removals for vehicle routing problems,” *Transportation Science*, vol. 54, no. 2, pp. 417–433, 2020.
- [13] T. Vidal, T. G. Crainic, M. Gendreau, and P. C. Prins, “A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows,” *Computers & Operations Research*, vol. 40, no. 1, pp. 475–489, 2013.
- [14] H. Lu, X. Zhang, and S. Yang, “A learning-based iterative search for the

- vehicle routing problem,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 10, pp. 4348–4362, 2020.
- [15] S. Son, J. Kim, and J. Park, “Learning to destroy and repair: A reinforcement learning approach for combinatorial optimization,” *IEEE Access*, vol. 11, pp. 12 845–12 856, 2023.
- [16] A. Hottung and K. Tierney, “Neural large neighborhood search for the capacitated vehicle routing problem,” in *Proc. 24th European Conference on Artificial Intelligence (ECAI)*, 2020, pp. 443–450.
- [17] P. Shaw, “Using constraint programming and local search methods to solve vehicle routing problems,” in *Proc. CP 1998*, ser. LNCS, vol. 1520. Springer, 1998, pp. 417–431.
- [18] D. Pisinger and S. Ropke, “Large neighborhood search,” in *Handbook of Metaheuristics*. Springer, 2010, pp. 399–419.
- [19] Y. Nagata and O. Bräysy, “A powerful route minimization heuristic for the vehicle routing problem with time windows,” *Operations Research Letters*, vol. 37, no. 5, pp. 333–338, 2009.
- [20] Y. Nagata, O. Bräysy, and W. Dullaert, “A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows,” *Computers & Operations Research*, vol. 37, no. 4, pp. 724–737, 2010.
- [21] Y. Rochat and E. D. Taillard, “Probabilistic diversification and diversification in local search for vehicle routing,” *Journal of Heuristics*, vol. 1, no. 1, pp. 147–167, 1995.
- [22] G. R. Alvarenga, G. R. Mateus, and G. de Azambuja Silveira, “A genetic and set partitioning approach for the vehicle routing problem with time windows,” *Computers & Operations Research*, vol. 34, no. 10, pp. 2991–3004, 2007.
- [23] A. Subramanian, P. H. V. Penna, E. Uchoa, and L. S. Ochi, “A hybrid algorithm for the vehicle routing problem with simultaneous pickup and delivery,” *Computers & Operations Research*, vol. 40, no. 1, pp. 468–477, 2013.
- [24] R. Turkeš, K. Sörensen, and L. M. Hvattum, “Meta-analysis of adaptive large neighborhood search,” *European Journal of Operational Research*, vol. 292, no. 2, pp. 423–442, 2021.
- [25] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, “Neural combinatorial optimization with reinforcement learning,” *arXiv preprint arXiv:1611.09940*, 2016.
- [26] E. Khalil, H. Dai, Y. Zhang, B. Dilkina, and L. Song, “Learning combinatorial optimization algorithms over graphs,” in *Advances in NeurIPS*, vol. 30, 2017, pp. 6348–6358.
- [27] C. K. Joshi, T. Laurent, and X. Bresson, “An efficient graph convolutional network technique for the travelling salesman problem,” *arXiv preprint arXiv:1906.01227*, 2019.
- [28] Y. Bengio, A. Lodi, and Y. Prouvost, “Machine learning for combinatorial optimization: A methodological tour d’horizon,” *European Journal of Operational Research*, vol. 290, no. 2, pp. 405–421, 2021.
- [29] A. Syberfeldt and T. Bäck, “Reinforcement learning for parameter control in metaheuristics,” in *Proc. IEEE Congress on Evolutionary Computation (CEC)*, 2009, pp. 2526–2533.
- [30] X. Chen and Y. Tian, “Learning to perform local rewrite for combinatorial optimization,” in *Advances in NeurIPS*, vol. 32, 2019, pp. 8484–8494.
- [31] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. 16th ICML*, 1999, pp. 278–287.
- [32] Z. Wang et al., “Dueling network architectures for deep reinforcement learning,” in *Proc. 33rd ICML*, vol. 48, 2016, pp. 1995–2003.
- [33] Q. Huangfu and J. A. J. Hall, “Parallelizing the dual revised simplex method,” *Mathematical Programming Computation*, vol. 10, no. 1, pp. 119–142, 2018.
- [34] M. W. Savelsbergh, “The vehicle routing problem with time windows: Minimizing route duration,” *ORSA Journal on Computing*, vol. 4, no. 2, pp. 146–154, 1992.
- [35] P. Toth and D. Vigo, “The granular tabu search and its application to the vehicle-routing problem,” *INFORMS Journal on Computing*, vol. 15, no. 4, pp. 333–346, 2003.
- [36] H. Gehring and J. Homberger, “A parallel hybrid evolutionary metaheuristic for the vehicle routing problem with time windows,” in *Proc. 5th Int. Conf. Parallel Comput. (Euro-Par)*, 1999, pp. 577–584.
- [37] B. Lin et al., “Neural combinatorial optimization with reinforcement learning on large-scale vehicle routing problems,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12 831–12 842, 2021.
- [38] J.-Y. Audibert, R. Munos, and C. Szepesvári, “Exploration–exploitation tradeoff using variance estimates in multi-armed bandits,” *Theoretical Computer Science*, vol. 410, no. 19, pp. 1876–1902, 2009.
- [39] P. Auer, N. Cesa-Bianchi, and P. Fischer, “Finite-time analysis of the multiarmed bandit problem,” *Machine Learning*, vol. 47, no. 2, pp. 235–256, 2002.
- [40] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [41] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double q-learning,” in *Proc. 30th AAAI Conf. Artificial Intelligence*, 2016,

- pp. 2094–2100.
- [42] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, “Prioritized experience replay,” in *International Conference on Learning Representations (ICLR)*, 2016.
- [43] R. S. Sutton, D. Precup, and S. Singh, “Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning,” *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.
- [44] SINTEF Optimization, “SINTEF VRPTW benchmark portal: Best known solutions for Homberger extended instances,” <https://www.sintef.no/projectweb/top/vrptw/homberger-benchmark/>, 2024.



HUYNH NHAT HUY is currently pursuing the B.S. degree in Computer Science with the Faculty of Information Technology, Ton Duc Thang University, Ho Chi Minh City, Vietnam. His research interests include deep reinforcement learning, combinatorial optimization, metaheuristics, and learning-augmented algorithms.



THI-LINH HO received the B.S. degree from Nong Lam University in 2009, the M.S. degree from Ho Chi Minh City University of Technology in 2012, and the Ph.D. degree in Computer Science from Ton Duc Thang University, Vietnam, in 2024. She is currently a Lecturer with the Faculty of Information Technology, Ton Duc Thang University. Her research interests include machine learning, intelligent information systems, and combinatorial optimization.



NGUYEN NHAT HUY received the B.S. degree in Computer Science from Ton Duc Thang University, Ho Chi Minh City, Vietnam, in 2026. His research interests focus on speech and language processing, generative AI, deep reinforcement learning, and combinatorial optimization.



NGUYEN THI BAO TRAN received the B.S. degree in Software Engineering from Ton Duc Thang University, Ho Chi Minh City, Vietnam, in 2026. Her current research interests include speech and language processing, generative AI, deep reinforcement learning, and combinatorial optimization.

...